

An efficient explicit implementation of a near-optimal quantum algorithm for simulating linear dissipative differential equations

I. Novikau¹ and I. Joseph¹

Lawrence Livermore National Laboratory, Livermore, California 94550, USA

(*Electronic mail: joseph5@llnl.gov)

(Dated: 17 April 2026)

We propose an efficient block-encoding technique for the implementation of the Linear Combination of Hamiltonian Simulations (LCHS) for simulating dissipative initial-value problems. This algorithm approximates a target nonunitary operator as a weighted sum of Hamiltonian evolutions, thereby emulating a dissipative problem by mixing various time scales. We introduce an efficient encoding of the LCHS into a quantum circuit based on a simple coordinate transformation that turns the dependence on the summation index into a trigonometric function. Classically, this method is equivalent to the use of a highly accurate Fejér-Clenshaw-Curtis quadrature formula. Quantumly, this significantly simplifies block-encoding of a dissipative problem and allows one to perform an exponential number of Hamiltonian simulations by a single Quantum Signal Processing (QSP) circuit. The resulting LCHS circuit has high success probability and the selector scales logarithmically with the number of terms in the LCHS sum and linearly with time. Careful analysis of error convergence proves that this method is more efficient than other LCHS circuits that have recently appeared in the literature. We verify the quantum circuit and its scaling by simulating it on a digital emulator of fault-tolerant quantum computers and, as a test problem, solve the advection-diffusion equation. The proposed algorithm can be used for modeling a wide class of nonunitary initial-value problems including the Liouville equation with added dissipation and linear embeddings of nonlinear systems, such as the Koopman-von Neumann and Carleman embeddings.

I. INTRODUCTION

A. Motivation

Precise modeling of complex dynamics characterized by multiple scales in space and time needs high spatial resolution. As a result, accurate simulations of many classical problems of practical interest require large amounts of numerical resources. This significantly limits the number of problems that can be simulated on classical computers, and it seems suitable to consider quantum computers (QCs) as possible candidates for modeling these sorts of problems. By leveraging quantum effects such as entanglement and superposition of quantum states, QCs can process exponentially many complex numbers in parallel. Moreover, in advanced quantum algorithms for solving differential equations, the number of operations grows polynomially or even exponentially more slowly than in the corresponding classical methods. Yet, quantum computers can operate only with linear unitary operators that makes quantum simulations of dissipative problems challenging.

To solve dissipative differential equations, one can transform them into a system of linear equations¹ $M\psi = b$ with a dilated matrix M . The scaling of the matrix condition number κ_M on the integration time depends on the Courant-Friedrichs-Lewy convergence condition and, for instance, is quadratic for the advection-diffusion or heat equations,² which have second-order spatial derivatives. The linear system can be solved by using a broad variety of efficient Quantum Linear System Algorithms (QLSAs) such as those based on variable-time amplitude amplification^{3–5} or discrete adiabatic theorem,^{6,7} whose query complexity is linear with the condition number. More precisely, the number of calls to the oracle encoding M and to the oracle encoding the right-hand-

side vector b is linear with κ_M .

Another option is the time-marching (TM) methods where one of the key issues is the decreasing success probability in time. In Ref. 8, it was proposed to use a combination of the uniform singular value amplification and compression gadgets to guarantee high success probability. The resulting algorithm scales quadratically with the simulated time. In Ref. 9, it was shown how to solve the advection-diffusion equation with the TM technique having linear complexity with time and high success probability. However, it is not clear whether this method would preserve the near-optimal scaling if the normalized advection-like component of the TM matrix cannot be block-encoded keeping the spectral norm of the encoded matrix equal to one. Also, explicit and implicit hybrid classical-quantum TM methods were proposed in Ref. 10. The TM algorithm was also used for modeling the Carleman linearization of the advection-diffusion-reaction equation.¹¹

One can also use hybrid classical-quantum methods for solving dissipative problems on quantum computers as proposed, for instance, in Ref. 12 and 13. Yet, the hybrid methods usually suffer from the bottleneck related to the repetitive input/output (I/O) data transfer between classical and quantum processors.

An alternative approach is based on recasting a nonunitary operator associated with dissipative dynamics into a combination of unitary evolution operators integrated over an additional Fourier space. This technique was initially proposed in Refs. 14–18 and is called “Schrödingerization” or Linear Combination of Hamiltonian Simulations (LCHS). The LCHS algorithm has an advantage over QLSAs in its higher success probability, which leads to a reduced number of calls to the initialization circuit. Furthermore, the encoding of the LCHS algorithm is more straightforward compared to that of

QLSAs.

We developed an efficient explicit implementation of the original LCHS algorithm¹⁸ and simulated its numerical performance in Ref. 19. However, we found that achieving modest % levels of precision required a fairly large ancillary register for the construction of the kernel (corresponding to parameters $n_k \geq 7$ and $N_k \geq 128$ in the following). The original LCHS algorithm was significantly improved by An, Childs, and Lin 20 by exponentially reducing its query complexity with respect to the error, ϵ_{LCHS} , to near-optimal scaling with precision, set out in Theorem 1 below. Hence, our work here is motivated by the practical question of whether a more efficient LCHS implementation that has better dependence on precision can offer a significant savings in resources.

Hence, we test the new LCHS algorithm on the key point example of the linear advection-diffusion equation. This is a particularly important example of a dissipative differential equation because it is used to model physical processes in many different fields including astronomy, astrophysics, biology, chemistry, condensed matter physics, finance, fluid dynamics, and plasma physics. In many contexts, the linearized version of advection-diffusion results from applying perturbation theory to a nonlinear problem, such as the Navier-Stokes equations or the Boltzmann kinetic equation. Yet, Liouville's theorem implies that the first-principles equations of motion have no nonlinearity for conservation of probability at either the classical or quantum mechanical level.

Hence, the linear advection-diffusion equation is also important for describing the first-principles probabilistic formulation of classical and semiclassical dynamics.²¹ This approach can be utilized to develop efficient quantum algorithms for evolving the probability distribution function for nonlinear dynamics based on the unitary Koopman-von Neumann (KvN) and Koopman-van Hove equations.^{19,22,23} In fact, the main goal of 19 was to develop an explicit high-performance quantum algorithm for simulating the KvN equation. There, the approach relied on numerical dissipation, by adding either an upwind advection operator or a numerical diffusion operator to the generator, to eliminate numerical artifacts caused by the way in which perfectly unitary dynamics responds to sources and sinks. Thus, simulating the evolution of the probability distribution of nonlinear dynamics is a potentially important future application of the efficient LCHS methods that we derive here.

B. Main results

The main focus of our work is to provide an efficient quantum circuit encoding of the LCHS algorithm and to test its performance in practice for an important point-example: the advection-diffusion equation. Although the seminal works 18, 20, 24, and 25 offer an extensive analytical description of the LCHS method, they do not provide a systematic description of the LCHS circuit. Our recent work 19 clearly demonstrated that the same algorithm described analytically can be encoded in a variety of ways that can significantly affect not only the total number of gates in the circuit

but also the overall complexity of the encoded algorithm and its dependence on various parameters of the method under consideration.

In particular, the authors of Refs. 16, 18, 19, and 26 proposed constructing the core component of the LCHS/Schrödingerization algorithm as a sequence of controlled time-evolution operators. In contrast, in this work we reduce that component to a single Hamiltonian evolution implemented via a single QSP circuit. The proposed encoding significantly simplifies the LCHS circuit compared to previous works. More precisely, we demonstrate how a single block-encoding oracle can simultaneously represent an exponential number of matrices, enabling an exponential number of quantum Hamiltonian simulations (QHS) to be performed in parallel using a single QSP circuit.

The encoding used here builds on the coordinate transformation proposed in our previous work.¹⁹ Here, we report on our discovery of how to leverage this transformation by incorporating it into the qubitization operator of the QSP circuit. This represents a significant improvement over the previous approach by eliminating the need for trotterization and significantly simplifying the selector circuit. We also analyze the complexity of the LCHS algorithm based on our encoding technique and show that, unlike the analysis in Refs. 18 and 19, it eliminates the dependence of algorithmic accuracy on the trotterization step and matrix commutators. Most importantly, the proposed encoding improves the scaling with respect to the simulated time.

We simulate the LCHS circuit on a digital emulator^{27–29} of fault-tolerant quantum computers and verify the circuit by modeling the advection-diffusion equation. In these simulations, we demonstrate the near-optimal scaling of the improved LCHS algorithm with precision, its high success probability, and how its accuracy depends on key parameters such as the number of points in Fourier space and the length of the simulated time interval for a key numerical point example. In the interest of open and reproducible science, all source files used in the simulations are available in Ref. 29 to allow the interested reader to reproduce our results.

Thus, in this work, we significantly improve on LCHS implementations in several important ways:

1. By using the recently proposed class of kernels in 20, we obtain near-optimal convergence with precision.
2. We use a simple sinusoidal coordinate transformation to easily block-encode both the Hermitian and non-Hermitian parts of the operator and eliminate the need for trotterization in 19.
3. Thanks to this simplification, we can perform all necessary Hamiltonian simulations with a single quantum signal processing³⁰ (QSP) circuit, requiring only two ancillae (without including block-encoding).
4. Our integration method is equivalent to Fejér-Clenshaw-Curtis (FCC) quadrature, and Lemma 2 shows that, for periodic and analytic functions, FCC is equivalent to Chebyshev-Gauss-Lobatto (CGL)

quadrature at the Chebyshev extrema. Thus, it has excellent convergence properties, especially in the exponentially convergent regime where it is optimally employed.

5. While the complexity analysis has recently been improved by focusing on the global convergence afforded by analytic function approximations^{24,25}, Lemma 3 proves that this is generically only an intermediate asymptotic for kernels of interest. Hence, regimes with power law scaling and nonlinear aliasing of the two factors in the LCHS integral require more careful consideration.
6. We perform several additional optimizations to reduce the overall number of ancillae.
7. Theorem 4 proves that the FCC quadrature method has better error convergence, and, hence, better overall complexity than other LCHS circuit implementations proposed in the literature 18, 20, and 24.

Taken together, this series of improvements represents a significant advance over alternate suggested implementations of the LCHS circuit to date.

Achieving this wide range of improvements for a practical numerical example is important step forward for LCHS methods. Because our goal is to simulate the numerical performance, we have to focus on reducing the number of ancillary qubits as much as reasonably possible so that the problem instance fits within the memory of the classical computer that emulates the quantum circuit. Moreover, this work and our previous work 19 are the only in-depth numerical investigations of the numerical performance of the algorithm for the key point example of the advection-diffusion equation. Because it is often difficult to perform a complete cost analysis for a complex problem, our work demonstrates the importance of performing careful numerical calculations that involve all key steps in the analysis.

While the original works 18 and 20 suggested using both QSP and quantum singular value transformation³¹ (QSVT) for different aspects of an implementation of LCHS, they did not present an explicit block-encoding of the entire circuit, including the most important part, the selector. In Ref. 20 (Section 4.2.1), a tentative description of an explicit circuit construction was proposed, but it is much more complicated than our version since it is based on composite Gaussian quadrature and their circuit includes several extra ancillary registers. Moreover, 20 explains that the most crucial step in LCHS is the implementation of the selector which is completely and definitively covered in our work.

Quite recently, after our work was completed and submitted, new approximate LCHS kernels that achieve the optimal scaling with precision for a linear differential equation solver³², $\mathcal{O}(\log(\epsilon_{\text{LCHS}}^{-1}))$, were derived in Refs. 24 and 25. The key idea, originally explored in 33 for the purpose of approximating the exponential function applied to an operator, is to use an “analytic extension” of the kernel function, based on analytic approximations to the step function. While these newer works have made important contributions to understanding and improving the precision of approximate LCHS,

only Ref. 24 gives an explicit block-encoding for the selector. (Ref. 25 cites 20 for the explicit circuit.) The “multiplexed block-encoding” algorithm developed by Ref. 24 Lemma 18 is more complicated than the construction proposed here, and requires several subroutines and additional ancillary registers, in fact, perhaps doubling or even quadrupling the size of the register required for the kernel itself. Thus, the key idea of using the sinusoidal coordinate transformation to perform FCC quadrature in combination with QSP for QHS is a simple and elegant solution that leads to a valuable reduction in both classical and quantum complexity that is important for practical implementations.

This paper is organized as follows. In Sec. II A, we review the LCHS method with the improved kernels and explain in detail how this algorithm can be mapped onto a quantum circuit. Then, in Sec. III, we consider the convergence of errors and how this affects the complexity of the LCHS algorithm. Next, in Sec. IV, we test the LCHS circuit by simulating the advection-diffusion equation and analyze the algorithm scaling, precision, and the circuit success probability. Finally, in Sec. V, we summarize the main results of this paper.

II. EFFICIENT IMPLEMENTATION OF LINEAR COMBINATION OF HAMILTONIAN SIMULATIONS (LCHS)

A. Exact LCHS

We consider a linear differential equation with a non-Hermitian time-independent generator A

$$\partial_t \psi(t, x) = -A \psi(t, x). \quad (1)$$

Here, the evolution in time of the variable $\psi(t, x)$ is described by the nonunitary operator e^{-At} :

$$\psi(t, x) = e^{-At} \psi(0, x), \quad (2)$$

where $\psi(0, x)$ represents the initial conditions and $\psi(t, x)$ is either real or complex. If the problem represents a partial differential equation (PDE), we assume that it has been discretized with N_x grid points, and, hence, that the solution vector can be stored in quantum memory within a register of size $n_x = \log_2(N_x)$. We note that both LCHS and Schrödingerization techniques have been developed for cases where the generator is time-dependent and where there is an inhomogeneous forcing in time; i.e. an equation of the form $d\psi/dt = -A(t)\psi(t, x) + b(t)$, but, in this work, we focus on the time-independent homogeneous case alone.

The initial-value problem (2) can be solved by exact LCHS^{18,20} which represents the nonunitary operator e^{-At} as a weighted superposition of Hamiltonian evolutions integrated over an additional Fourier space

$$e^{-At} = \int_{\mathbb{R}} \frac{\xi(k)}{1 - ik} e^{-i(A_H + kA_L)t} dk, \quad (3)$$

where the original generator A has been separated into the Hermitian matrices

$$A = A_L + iA_H, \quad (4a)$$

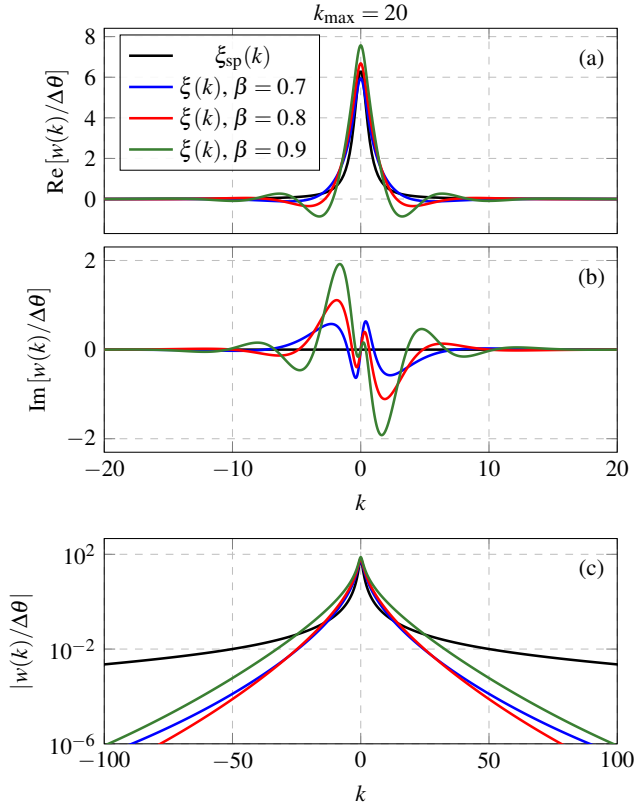


FIG. 1: Plots showing the real (a) and imaginary (b) components of the LCHS weights for the special case of the Cauchy kernel (8) (black line) and the improved kernel (5) with various β (colored lines). (c): Absolute values of the LCHS weights.

$$A_L = (A + A^\dagger)/2, \quad (4b)$$

$$A_H = (A - A^\dagger)/(2i). \quad (4c)$$

Equation (3) is valid as long as $A_L \geq 0$ is positive semi-definite. This can be extended to non-positive matrices, A_L , by applying the LCHS method to $A' = A + |\lambda|I$, where I is the identity matrix and $\lambda < 0$ is a lower bound for the smallest, i.e. largest in magnitude but negative, eigenvalue of A_L . To find the actual behavior in time, one must then rescale the results in time by multiplying by $e^{|\lambda|t}$ as a classical post-processing step.

An, Childs, and Lin 20 Result 1 (Theorem 6) proved that many choices of kernel are possible as long as they satisfy a few conditions: (i) decay $|k|^a |\xi| \leq C$ for some $a, C > 0$, (ii) normalization $\int \xi dk / (1 - ik) = 1$, (iii) continuity on the real line, and, (iv) analyticity in the lower half plane which allows one to close the contour integral in the lower half plane for $A_L t \geq 0$. However, their no-go result 20 Proposition 8 proves that, for kernels that satisfy these conditions and exactly satisfy Eq. (3), it is not possible to achieve the optimal scaling with precision, $\mathcal{O}(\log(\epsilon_{\text{LCHS}}^{-1}))$, expected for an ODE solver³². Yet, they showed that the family of kernels:

$$\xi(k) = \frac{1}{2\pi e^{-2\beta} \exp([1 + ik]^\beta)}, \quad (5)$$

with the real scalar $\beta \in (0, 1)$ lead to near-optimal dependence on precision in the form $\mathcal{O}(\log^{1/\beta}(\epsilon_{\text{LCHS}}^{-1}))$.

They also proved²⁰ that the exact LCHS method based on kernel (5) leads to a quantum linear differential equation solver that is nearly optimal in all respects. For clarity, we now recall their Result 3 (Corollary 17) for a homogeneous differential equation with time-independent generator, A , which follows from their Result 2 (Theorem 15 and Corollary 16), which treats the inhomogeneous and time-dependent case.

Theorem 1 (Near-Optimal LCHS Algorithm 20 Corollary 17). *There is a quantum LCHS algorithm based on kernel (5) that prepares the normalized solution of Eq. (1) with $\Omega(1)$ success probability and a flag indicating success that uses Q_A queries to the oracle encoding A and Q_{in} queries to the initial state preparation oracle, where*

$$Q_A = \mathcal{O}\left(\frac{\|\psi(0)\|}{\|\psi(t)\|} \|At\| \log^{1/\beta}(\epsilon_{\text{LCHS}}^{-1})\right) \quad (6)$$

$$Q_{in} = \mathcal{O}\left(\frac{\|\psi(0)\|}{\|\psi(t)\|}\right). \quad (7)$$

Proof. The LCHS integral can be approximated optimally using LCU³⁴ where the weight functions are computed optimally using QSVT³⁵ and the success probability is boosted with amplitude amplification³⁶. Given an efficient block-encoding of A_H and A_L , each Hamiltonian simulation step can be performed optimally using either QSP³⁰ and qubitization³⁵ or the truncated Dyson series method³⁷. $\square$

In contrast, the original LCHS method described in Ref. 18, which is equivalent to the mathematical formulation of the Schrödingerization algorithm in 38 employs the special case of the simple but suboptimal Cauchy kernel

$$\xi_{\text{sp}}(k) = \frac{1}{\pi(1 + ik)}. \quad (8)$$

Despite the simplicity, truncating the original LCHS integral at maximum values of $\pm k_{\text{max}}$ yields a truncation error of order $\epsilon_{\text{LCHS}} \sim \mathcal{O}(k_{\text{max}}^{-1})$, so the cost to achieve a given precision scales as $\mathcal{O}(\epsilon_{\text{LCHS}}^{-1})$. We tested the performance of this method in Ref. 19 for both advection-diffusion and upwind advection. While we validated the algorithm and our implementation, we found that, in practice, significant resources were required to reach a relative precision of order 10^{-2} . Thus, the main motivation for this work was to find an LCHS implementation with better performance for a practical numerical example.

One of our key contributions is the realization that to efficiently map the LCHS equation (3) onto a quantum circuit, it is beneficial to apply the following coordinate transformation:¹⁹

$$k = k_{\text{max}} \sin(\theta) \quad (9)$$

and to use this in the block-encoding of the Hamiltonian $C_k = kA_L + A_H$. This choice allows one to recast the dependence on the Fourier coordinate k as a trigonometric function, $\sin(\theta)$, which is much easier to block-encode than k itself, which technically requires implementing the $\arcsin(\theta)$ function.

Because $\arcsin(\theta)$ is not continuous, this implies that the Chebyshev coefficients only converge as $1/N_q$, where, here, N_q is the number of terms in the series. Hence, to achieve a relative error of order ϵ_{LCHS} , a large number of terms is required. While 39 and 40 have performed studies of the performance of this choice, and there may be methods for improvement using the “number operator” encoding of 24, the scaling appears to be suboptimal. In contrast, the coordinate transformation of Eq. (9) is represented exactly by a single term. Relative to our previous implementation¹⁹, this choice (1) eliminates the need for trotterization of kA_L and A_H and (2) significantly simplifies the implementation of the selector, S . (3) Finally, the choice of near-optimal kernel (5) allows our new implementation to achieve near-optimal dependence on precision.

The LCHS computation (3) of the nonunitary operator e^{-At} is exact when one works in infinite Fourier space. Yet, to simulate Eq. (2) numerically, one must truncate Fourier space, i.e. by imposing $|k| < k_{\text{max}}$ and by discretizing the space with a grid with $N_k = 2^{n_k}$ points. In order to approximate the integral, we discretize Fourier space through the transformation (9) via

$$k_j = k_{\text{max}} \sin(\theta_j), \quad (10)$$

where the discretized angle θ_j is

$$\theta_j = -\frac{\pi}{2} + j\Delta\theta, \quad \Delta\theta = \frac{\pi}{N_k - 1}, \quad j = 0, 1, \dots, N_k - 1. \quad (11)$$

Hence, one obtains a discretized version of the initial-value problem (2),

$$\psi(t, x) = U_{\text{LCHS}} \psi(0, x) + \epsilon_{\text{LCHS}}, \quad (12)$$

where ϵ_{LCHS} is the combination of discretization and truncation error. The discretized LCHS operator U_{LCHS} approximates the nonunitary operator e^{-At} as

$$e^{-At} \approx U_{\text{LCHS}} = \sum_{j=0}^{N_k-1} w_j V_j(t), \quad (13)$$

where each unitary V_j ,

$$V_j(t) = e^{-iC_j t}, \quad (14)$$

depends on the θ -dependent Hermitian matrix C_j ,

$$C_j = A_H + \sin(\theta_j) B_m, \quad (15a)$$

$$B_m = k_{\text{max}} A_L, \quad (15b)$$

and the sum (13) is weighted by the complex coefficients w_j ,

$$w_j = \frac{k_{\text{max}} \cos(\theta_j) \Delta\theta \xi(k_j)}{1 - ik_j}. \quad (16)$$

These weights become real when the Cauchy kernel (8) is applied. The shape of the LCHS weights w_j computed with the improved and Cauchy kernels, Eq. (5) and Eq. (8), correspondingly, are shown in Fig. 1. One can see that the real and imaginary components of the complex weights built with

the improved kernel ξ are functions of definite parity. This means that each component can be constructed by using a single QSVT circuit. Also, as can be seen from Fig. 1c, the improved kernel causes the weights to decay exponentially faster than in the special case (8). According to Ref. 20 and as is also demonstrated later in this paper, this results in the exponential decay of the truncation error ϵ_{LCHS} with k_{max} :

$$\epsilon_{\text{LCHS}} = \mathcal{O}\left(e^{-\mathcal{O}(k_{\text{max}}^\beta)}\right). \quad (17)$$

At the same time, the truncation error in the discretized LCHS computation (12) with the Cauchy kernel ξ_{sp} is only inversely proportional to k_{max} :^{18,19}

$$\epsilon_{\text{LCHS}} = \mathcal{O}(k_{\text{max}}^{-1}). \quad (18)$$

B. Comparison of Exact and Approximate LCHS

After this work was submitted, more general kernels that achieve more optimal scaling with precision were derived in Refs. 24 and 25. In fact, Ref. 33, previously explored the same idea of using an “analytic extension” of a Fourier series for the goal of computing the exponential function of an operator using the QSP-based algorithm they derived for constructing an arbitrary Fourier series of a block-encoded operator, and, studied the performance numerically.

The key to avoiding the no-go theorem of 20 is to search for kernels that approximate Eq. (3) to precision $\mathcal{O}(\epsilon_{\text{LCHS}})$. The simplest versions of such kernels approximate exponential decay in time with a function that is holomorphic in the entire complex plane (an entire function); i.e. they either approximate the functions $e^{-|x|}$ or $e^{\mp x} \Theta(\pm x)$ where $\Theta(x)$ is the step function, by using the error function, $\text{erf}(x)$, as an analytic approximation to the step function. While approximations of this type had already been explored in 33 and while the error analysis of 33 and 24 provide similar estimates, Refs. 24 and 25 made significant and valuable contributions in terms of analyzing the appropriate form of such approximations and the associated complexity in the context of approximate LCHS. Interestingly enough, an extensive search for optimality in 24 (see their Table 3) found that approximate LCHS kernels that were numerically optimized to have the best dependence on precision have the form of a rational function multiplied by the simple weight function and do not use the error function at all. We refer the reader to Refs. 24, 25, and 33 for explicit derivations of examples of these kernels as well as proofs of their existence and optimality.

While it is well beyond the scope of this work to test the performance of these new kernels, our implementation is agnostic of the particular form of the choice of kernel and will work equally well for these improved kernel choices. The main difference is that the dependence of the query complexity with precision goes from $\log^{1/\beta}(\epsilon_{\text{LCHS}}^{-1})$ to $\log(\epsilon_{\text{LCHS}}^{-1})$, where, for the example studied here and as first observed in 20, a near-optimal range is $\beta \in [0.7, 0.8]$, i.e. $1/\beta \in [1.25, 1.43]$.

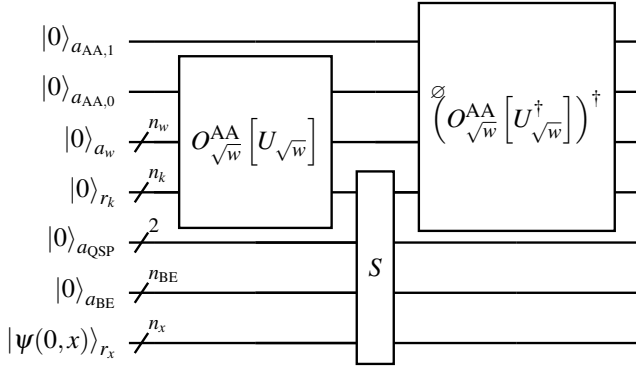


FIG. 2: LCU circuit for solving the LCHS equation (12). The selector is shown in Fig. 3. The $O_{\sqrt{w}}^{AA}$ operators perform amplitude amplification (AA) of the $U_{\sqrt{w}}$ operators which construct the weights via QSVT. These operators are explicitly defined in 19 (Fig. 6 and 7, respectively). The symbol $\emptyset$ indicates that the corresponding qubit is not used by the indicated subcircuit.

A relatively straightforward explicit block-encoding for the selector, which requires a block-encoding of $C_k = kA_L + A_H$, was given in Ref. 24. Their approach directly encodes the number state $\sum_k k|k\rangle$ and requires a second ancillary qubit register of size $n_k = \log_2(N_k)$, which doubles the size of the total ancillary memory register for LCHS to $\sim 2n_k$. This procedure requires an additional $2 \times \mathcal{O}(n_k)$ two-qubit gates to apply a unitary that constructs the state $k^{1/2}|k\rangle$ as well as the inverse, to ultimately apply the $k|k\rangle\langle k|$ operator. Then, one must perform a controlled SWAP of the two k registers at an additional cost of $\mathcal{O}(3n_k)$; however, for fixed hardware layout, the SWAP cost can be much higher. As discussed more fully in Sec. III C, the total ancillary register required for this part can be as large as $4n_k$, depending on whether the extra required ancillary registers can be reused.

In contrast, the coordinate transformation of Eq. (9) only requires $\mathcal{O}(1)$ ancillary qubits, and, as we shall see later, because it represents a smooth transformation of variables, it only has an $\mathcal{O}(1)$ impact on the cost of approximating the weight function using QSVT. Finally, the cost of constructing the $\sin(\theta)$ function (see Fig. 4c) is precisely n_k controlled rotations and 2 single-qubit rotations.

C. Explicit LCHS Circuit

1. Overview

An entire quantum linear differential equation solver algorithm (QLDS) based on LCHS consists of amplitude amplification (AA)³⁶ of the overall LCHS circuit and then a measurement of interest. As will be explained further below, AA is essential for the QLDS to solve a dissipative differential equation because the success probability, which tracks the amplitude, tends to decay in time. The measurement step is typically preceded by a series of state preparations required

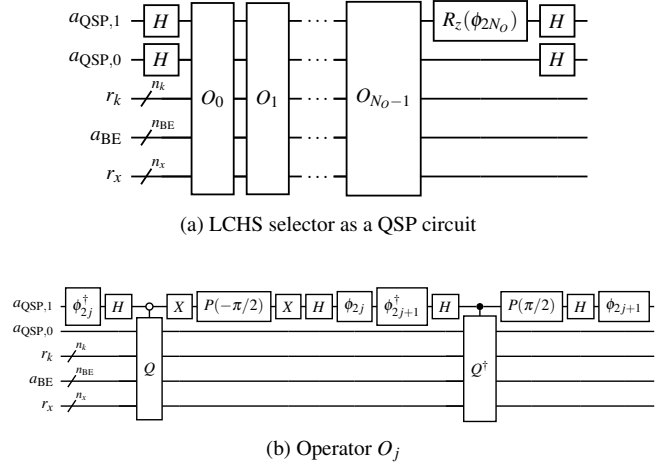


FIG. 3: (a) The selector circuit S for computing the N_k unitaries V_j [Eq. (14)]. The selector is represented by a single QSP^{30,35} circuit consisting of N_O operators O_j , where the number N_O depends on the simulated time interval t . (b) Circuit for the operators O_j for $j = 0, 1, \dots, N_O - 1$. Each O_j operator depends on two QSP angles, ϕ_{2j} and ϕ_{2j+1} . Here, the gates schematically denoted as ϕ_l indicate the rotations $R_z(\phi_l)$. The iterate Q is shown in Fig. 4

to measure the physical observable of interest, e.g. see 22 for an explicit discussion of how to measure an observable of interest, and typically also requires AA to boost the success probability and/or to accurately measure an amplitude. In this work, we only focus on an efficient construction and emulation of the core LCHS circuit alone because the AA and measurement steps are standard. Interestingly enough, for the numerical example over the specific time period studied below, we find that the overall success probability is actually ~ 0.2 , and, since LCHS terminates with flag indicating success, AA is not strictly required.

2. LCU implementation of LCHS

The discretized LCHS transformation is expressed in the form of a weighted sum, Eq. (13). As such, it can be mapped on a Linear Combination of Unitaries (LCU) circuit of the standard form

$$LCU = \left(O_{\sqrt{w}}^{AA}[U_{\sqrt{w}}^\dagger] \right)^\dagger \circ S \circ O_{\sqrt{w}}^{AA}[U_{\sqrt{w}}], \quad (19)$$

schematically shown in Fig. 2. There, the oracles $O_{\sqrt{w}}^{AA}[U_{\sqrt{w}}]$ and $\left(O_{\sqrt{w}}^{AA}[U_{\sqrt{w}}^\dagger] \right)^\dagger$ compute the LCHS weights (16), and the selector oracle S , shown in Fig. 3, computes the N_k unitaries V_j , Eq. (14), through a single QSP circuit, effectively in parallel. The selector uses the operators, O_j , shown in Fig. 3b, which contain the iterate Q shown in Fig. 4a. The iterate depends on the block-encoding of C_j given by U_C and $U_C^\dagger$. To encode C_j efficiently, one needs access to oracles U_{A_H} and U_{B_m} that block-encode A_H and $B_m = k_{\max}A_L$, as well as the

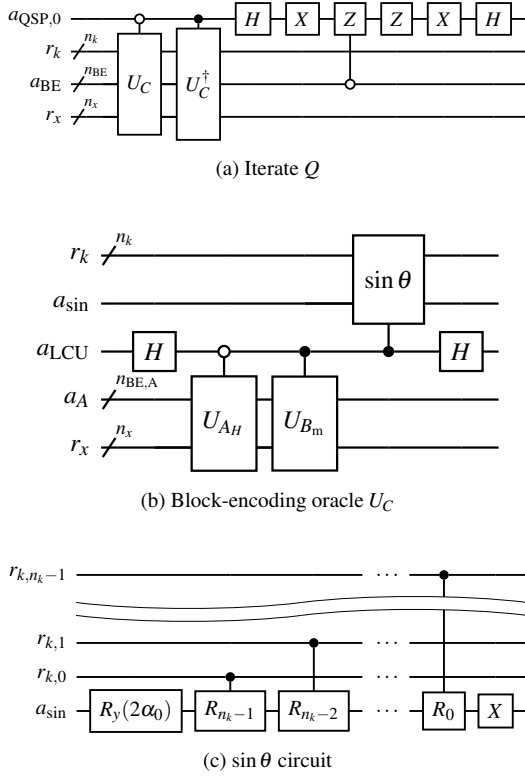


FIG. 4: (a) The iterate circuit Q used in the selector of Fig. 3b. Note that the qubitization is performed only with respect to the ancillary register a_{BE} independently of the register r_k , although the latter is also used by the block-encoding oracle U_C . (b) The block-encoding oracle U_C encoding C_j in Eq (15). The implementation of the subcircuits U_{A_H} and U_{B_m} depends on the structure of A . The registers a_A , a_{LCU} , and a_{sin} are different parts of the register a_{BE} used in Fig. 4a. (c) Circuit encoding the function $\sin(\theta_j)$ for $j = 0, 1, \dots, N_k - 1$. Here, $R_l = R_y(2\alpha_l/2^l)$, $\alpha_0 = -\pi/2$, and $\alpha_1 = |\alpha_0|N_k/(N_k - 1)$.

$\sin(\theta)$ circuit. All necessary subcircuits are explicitly shown in Fig. 4. The oracles U_{A_H} and U_{B_m} for our particular case of the linear advection-diffusion equation are given in Fig. 5. The resources used by our LCHS circuit are summarized in Tables I and II.

The register r_k is used to encode the dependence on the Fourier coordinate k_j , expressed as the sine function of the angles θ_j according to Eq. (10). The ancillary qubits a_{QSP} and a_{BE} are used for the construction of the selector and for block-encoding the matrices C_j , Eq. (15). The ancillary register a_w with n_w qubits is used by both oracles $O_{\sqrt{w}}^{AA}[U_{\sqrt{w}}]$ and $(O_{\sqrt{w}}^{AA}[U_{\sqrt{w}}^\dagger])^\dagger$ for computing the LCHS weights. In addition, the oracles $O_{\sqrt{w}}^{AA}[U_{\sqrt{w}}]$ and $(O_{\sqrt{w}}^{AA}[U_{\sqrt{w}}^\dagger])^\dagger$ use the ancillae $a_{AA,0}$ and $a_{AA,1}$, correspondingly, for AA of the computed weights. Finally, the register r_x encodes the initial condition $\psi(0, x)$, which must be precomputed by an additional initialization subcircuit. The same register outputs the result, $\psi(t, x)$, entangled with the zero state of all ancillary qubits

and the register r_k .

3. Weights prepared by QSVT

The complex LCHS weights can be computed by two QSVT circuits combined by an LCU. There, each QSVT can be implemented in the manner explained in detail in Ref. 19 and will require $n_w = 3$ qubits in register a_w , taking into account the LCU procedure necessary to combine complex weights from real functions returned by QSVT. In particular, the oracle $O_{\sqrt{w}}^{AA}[U_{\sqrt{w}}]$ includes as a subcircuit the oracle $U_{\sqrt{w}}$, which computes the complex state

$$U_{\sqrt{w}}|0\rangle_{r_k} = \sum_j \sqrt{w_j}|j\rangle_{r_k} \quad (20)$$

by using the combined QSVT-LCU circuit. An alternative approach for the computation of $\sqrt{w_j}$ is a tensor-network-based technique described in Ref. 26. In our numerical emulations, we use a direct exact brute-force computation of the LCHS weights to minimize the number of ancillary qubits in the circuit. In this case, only $n_w = 1$ ancilla is needed in register a_w , and the oracle $U_{\sqrt{w}}$ is constructed as a sequence of N_k combined rotations $R_y(\phi_{y,j})R_z(\phi_{z,j})$ where $\phi_{y,j} = 2\arccos(|\sqrt{w_j}|)$ and $\phi_{z,j} = -2\arg(\sqrt{w_j})$.

After the weight computation, one should apply AA to amplify the success probability of the oracle $U_{\sqrt{w}}$, thereby boosting the success probability of the entire LCHS circuit 2. The amplification is implemented by the oracle $O_{\sqrt{w}}^{AA}[U_{\sqrt{w}}]$ and can be accomplished by using the standard AA technique.³⁶ The need for AA in these steps was explained in detail in Ref. 19. In particular, AA requires $2N_{AA}$ repetitions of the subcircuit $U_{\sqrt{w}}$. For instance, for $k_{\max} = 40$, one has $N_{AA} = 30$ and $N_{AA} = 43$ for $n_k = 11$ and $n_k = 12$, correspondingly. An important remark here is that AA does not concern the selector S which is the most computationally intensive component of the LCHS circuit. Therefore, one performs only $\mathcal{O}(N_{AA})$ repetitions of $U_{\sqrt{w}}$ without touching the selector. Apart from this, since S makes the dominant contribution to the complexity of the LCHS circuit and scales with time, the choice in the implementation of $U_{\sqrt{w}}$ has a small effect on the length of the entire LCHS circuit.

4. Selector implementation with a single QSP circuit

The selector S is implemented as a single QSP^{30,35} circuit shown in Fig. 3 which is only possible due to the coordinate transformation (9). In particular, the QSP circuit computing the unitaries V_j depends on the block-encoding oracle U_C shown in Fig. 4b encoding the Hermitian C_j , Eq. (15), where C_j is normalized as

$$C_j \rightarrow C_j/\|C_{\max}\|, \quad (21a)$$

$$\|C_{\max}\| = \|A_H + k_{\max}A_L\|. \quad (21b)$$

Each matrix C_j is represented by a sum of two Hermitians, A_H and B_m , and depends on the sine function $\sin(\theta_j)$. Therefore, the oracle U_C is constructed as an LCU combining two

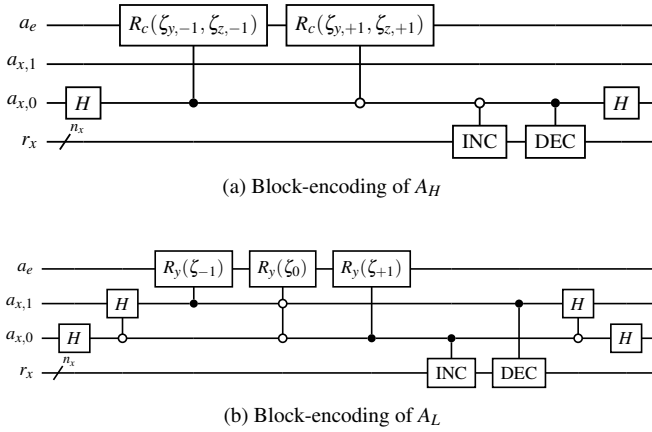


FIG. 5: (a) The oracle encoding the matrix A_H , Eq. (4), for the advection-diffusion Eq. (99). (b) The oracle encoding the matrix A_L . The circuits for the incrementor (INC) and decrementor (DEC) can be found in Ref. 41, Figure 14; these circuits scale as $\mathcal{O}(n_x)$. The gate schematically denoted by R_c is described in Eq. (102). The ζ_j parameters of the rotation operators are described in Eq. (103).

subcircuits, U_{A_H} and U_{B_m} , and an extra subcircuit computing the sine function, Fig. 4c. In particular, the coordinate transformation (9) is necessary to significantly simplify the encoding of the dependence on the Fourier coordinate k by using the subcircuit 4c. The implementation of the block-encoding oracles U_{A_H} and U_{B_m} is problem-specific and depends on the structure of the original non-Hermitian generator A used in Eq. (1).

Our proposed implementation of the selector is much more compact than that given in previous works such as Refs. 16, 19, and 26. In particular, instead of using trotterization where each time step is simulated by a sequence of n_k controlled Hamiltonian simulations, our encoding performs a single Hamiltonian evolution. This is achieved by hiding the dependence on the Fourier coordinate k within the iterate, Q . As shown in Sec. III A 3, this eliminates the dependence of the overall accuracy on the trotterization step and the commutator of the matrices A_L and A_H . Thus, the selector achieves linear scaling with time.

5. Block-Encoding of Hamiltonian $C = A_H + kA_L$

Efficient block-encodings are necessary for quantum advantage and block-encodings for many important PDEs have been derived.^{19,42} The block-encoding oracles U_{A_H} and U_{B_m} can be constructed following the procedure thoroughly described in Refs. 19, 43, and 44. Our definition of block-encoding is standard and is, for example, given in the seminal work Ref. 31 Def. 24. Let us assume that we need to encode a matrix M . Then, the goal of the block-encoding is to construct a circuit represented by the unitary U_M such that

$$U_M = \begin{pmatrix} \frac{M}{\|M\|\zeta} & \cdot \\ \cdot & \cdot \end{pmatrix}, \quad (22)$$

where $\|M\|$ is the matrix spectral norm, the scalar ζ is determined by the matrix sparsity ζ' , which is the maximum number of nonzero elements in any row or column of the matrices A_L and A_H . The upper bound of ζ can be estimated as

$$\zeta = 2^{\lceil \log_2 \zeta' \rceil}. \quad (23)$$

The oracle U_M acts on the registers a_A and r_x :

$$\left| \frac{M_{rc}}{\|M\|\zeta} - \langle c|_{r_x} \langle 0|_{a_A} U_M |r\rangle_{r_x} |0\rangle_{a_A} \right| \leq \epsilon_{BE}, \quad (24)$$

where r and c are the row and column indices, respectively, of a nonzero element of the matrix M , and ϵ_{BE} is the block-encoding error.

6. Qubitization

Qubitization is used to transform each eigenvalue of the block-encoded matrix C_j within its own Hilbert subspace. Thus, all matrix eigenvalues are transformed in their own disjoint two-dimensional subspaces which allows us to build different powers of the encoded matrix. In this way, we obtain a Grover-like search parallelized over all matrix eigenvalues. It is important to emphasize that, in this paper, we employ fully coherent QSP in the formulation described in Ref. 35. Thus, there is no impact on the success probability and this step does not require AA.

The oracle U_C is a part of the iterate Q shown in Fig. 4a, which performs qubitization.³⁵ This oracle can be constructed as

$$Q = (U_R \otimes I_{r_x, r_k}) U_C', \quad (25)$$

where U_R is the reflector operator

$$U_R = 2|+\rangle_{a_{QSP,0}} |0\rangle_{a_{BE}} \langle 0|_{a_{BE}} \langle +|_{a_{QSP,0}} - I_{a_{QSP,0}, a_{BE}}, \quad (26)$$

and U_C' is a combination of two controlled block-encoding oracles U_C

$$U_C' = |0\rangle_{a_{QSP,0}} \langle 0|_{a_{QSP,0}} \otimes U_C + |1\rangle_{a_{QSP,0}} \langle 1|_{a_{QSP,0}} \otimes U_C^\dagger. \quad (27)$$

Also, I_{r_x, r_k} is the unit operator that acts on the registers r_x and r_k , and $I_{a_{QSP,0}, a_{BE}}$ is the unit operator that acts on the registers $a_{QSP,0}$ and a_{BE} .

More precisely, qubitization is implemented by using the reflection operator U_R , represented by the last six gates in Fig. 4a. The latter must perform the reflection with respect to any ancillary qubit state orthogonal to the state entangled with the encoded matrices C_j . In our case, each matrix C_j for a particular j is entangled with the state $|j\rangle_{r_k} |0\rangle_{a_{BE}}$. Since we want to implement the qubitization for the matrices C_j with all $j = 0, 1, \dots, N_k - 1$ in parallel, we perform the reflection only with respect to $|0\rangle_{a_{BE}}$ independently of the state of the register r_k . This ensures that the QSP circuit computes the unitaries V_j for all j at once. In other words, the coordinate transformation (9) and the oracle 4 allow us to perform N_k Hamiltonian simulations in parallel by using a single QSP circuit.

Figure	Circuit	Purpose	Complexity Scaling
2	LCHS	LCHS implemented as an LCU circuit, Eq. (13)	$\mathcal{O}\left(\frac{\ \psi(0)\ }{\ \psi(t)\ }(Q_{\text{sel}} + Q_w)\right)$, Eq. (29)
3a	S	selector implemented as a QSP ³⁵ circuit a step in the QSP procedure qubitization iterate ³⁵	$Q_{\text{sel}} = \mathcal{O}(Q_{\text{BE}}[k_{\text{max}}\ \bar{C}_{\text{max}}\ t + Q_{\varepsilon, \text{QSP}}])$, Eq. (39)
3b	O_j		
4a	Q		
4b	U_C		
4c	$\sin(\theta)$		
5a	U_{A_H}	encode A_H , Eq. (4)	$Q_{\text{BE}} = \mathcal{O}(n_k + \text{poly}(n_x, \zeta'))$, Eq. (38)
5b	U_{B_m}	encode B_m , Eq. (15b)	$\mathcal{O}(n_k)$
Fig. 6 in Ref. 19	$O_{\sqrt{w}}^{\text{AA}}$ in circuit 2	amplitude amplify weights	$\mathcal{O}(\text{poly}(n_x, \zeta'))$
Fig. 7 in Ref. 19	$U_{\sqrt{w}}$ in circuit 2	encode weights w_j	$\mathcal{O}(\text{poly}(n_x, \zeta'))$

TABLE I: LCHS subcircuits, their purpose, and scaling. The numerical implementation of the entire LCHS circuit can be found in Ref. 29. The implementation of the block-encoding oracles U_{A_H} and U_{B_m} (Fig. 5) is specific to the linear uniform advection-diffusion problem and will differ for other problems. The structure of the rest of the LCHS circuit (Figs. 2, 3, and 4) is problem-independent.

Circuits	Register	Qubits	Use	Purpose
2-5	r_x	n_x	I/O	input $\psi(0, x)$ and output $\psi(t, x)$ entangled with the zero state of all ancillae
2	a_w	$n_w = 3$	ancilla	LCU-QSVT computing weights w_j
2	a_{AA}	$n_{\text{AA}, w} = 2$	ancilla	amplitude amplification of QSVT
2 - 4	r_k	n_k	ancilla	encode the integer j , Eq. (11), to address angle θ_j
2 - 4a	a_{QSP}	2	ancilla	construct the QSP and the iterate circuits for the implementation of selector S
2 - 4a	$a_{\text{BE}} = a_{\text{sin}} + a_{\text{LCU}} + a_A$	$n_{\text{BE}} = 2 + n_{\text{BE}, A}$	ancilla	block-encode C_j
4b and 4c	a_{sin}	1	in ancilla a_{BE}	for computing $\sin(\theta_j)$
4b	a_{LCU}	1	in ancilla a_{BE}	for computing LCU, i.e. the sum operation in Eq. (15)
4b	$a_A = a_e + a_x$	$n_{\text{BE}, A}$	in ancilla a_{BE}	block-encoding B_m and A_H
5	a_e	1	in ancilla a_A	for computing nonzero elements in the matrices A_H and B_m
5	a_x	2	in ancilla a_A	encoding column indices of nonzero elements in the matrices A_H and B_m

TABLE II: Listing of the size and purpose of LCHS state and ancillary registers. Aside from the register r_k , $9 + n_{\text{BE}, A}$ ancillae are required for this implementation of LCHS. In the case of the linear uniform advection-diffusion equation, $n_{\text{BE}, A} = 3$. This does not account for additional registers that may be required for pre-processing steps such as the initialization of the circuit and the computation of initial conditions or for post-processing steps such as amplitude estimation and measurement of physical observables.

III. LCHS CONVERGENCE & COMPLEXITY

$$= \varepsilon_{\text{cont}} + \varepsilon_{\text{trunc}} + \varepsilon_{\text{quad}} + \varepsilon_{\text{HS}} + \varepsilon_w. \quad (28)$$

A. Overall Complexity

1. Error Components

The total error for the LCHS algorithm has multiple components, $\varepsilon_{\text{LCHS}} = \varepsilon_{\text{cont}} + \varepsilon_{\text{disc}}$. First, an approximate LCHS method has the error, $\varepsilon_{\text{cont}}$, because the choice of continuous kernel only approximates exact LCHS. Second, the error associated with discretizing the integral, $\varepsilon_{\text{disc}} = \varepsilon_{\text{trunc}} + \varepsilon_{\text{quad}}$, has two parts, the truncation error, $\varepsilon_{\text{trunc}}$, due to cutting off the integral at $\pm k_{\text{max}}$, and the quadrature error, $\varepsilon_{\text{quad}}$, due to discretizing the integral with a finite set of points. Finally, another source of error results from the subroutine that computes the integrand, $\varepsilon_{\text{func}} = \varepsilon_{\text{HS}} + \varepsilon_w$, which, in turn, is the sum of errors in Hamiltonian simulation, ε_{HS} , and the computation of the weight function, ε_w . Thus, the total error can be summarized as

$$\varepsilon_{\text{LCHS}} = \varepsilon_{\text{cont}} + \varepsilon_{\text{disc}} + \varepsilon_{\text{func}}$$

Clearly, one can achieve a given overall LCHS error by requiring each component to satisfy $\varepsilon_* \leq \varepsilon_{\text{LCHS}}/m_\varepsilon$, where m_ε is the number of terms in the total error, e.g. $m_\varepsilon = 5$ for the expression above. Finally, for the example studied here, we use an exact near-optimal LCHS method, so $\varepsilon_{\text{cont}} = 0$ and, thus, $m_\varepsilon = 4$ in the case of exact LCHS.

2. Complexity of the Entire LCHS Circuit

The complexity of the entire LCHS circuit is set by the adding the cost of the selector, Q_{sel} to the cost of computing the weights, Q_w , via QSVT, and the cost needed to amplitude amplify the success probability. Thus, the scaling of the entire LCHS algorithm is

$$Q_{\text{LCHS}} = \mathcal{O}\left(\frac{\|\psi(0)\|}{\|\psi(t)\|}(Q_{\text{sel}} + Q_w)\right), \quad (29)$$

where the term Q_{sel} , Eq. (39), is the dominant one because of its dependence on the simulated time t . Here, the multiplicative factor $\|\psi(0)\|/\|\psi(t)\|$ takes into account the decrease of the LCHS success probability due to the decay of the simulated signal ψ in time. This happens because of the dissipation in the considered nonunitary problem (1).

For the optimal quadrature methods first proposed in this work and discussed further below, the quantum cost of constructing the weights, Q_w , is only weakly dependent on time. Thus, it is subdominant to the cost of the selector. In contrast, as will be explained further below, if trotterization is used^{18,19} or if composite Gaussian quadrature is used²⁰, the cost of Q_w increases superlinearly with time.

3. Selector Complexity with Efficient Block-Encoding

The selector S makes the main contribution to the cost of the LCHS algorithm, because S scales with time. In particular, since S is implemented by using a QSP circuit, its query complexity in terms of the number of calls to the block-encoding oracle U_C is

$$\mathcal{O}(\|C_{\text{max}}t\| + \log_2 \epsilon_{\text{QSP}}^{-1}), \quad (30)$$

where the simulated time t is multiplied by the spectral norm $\|C_{\text{max}}\|$ because the encoded matrices C_j are normalized according to Eq. (21). An important remark here is that $\|C_{\text{max}}\|$ can grow linearly with k_{max} . To show this dependence explicitly, we rewrite Eq. (30) as

$$\mathcal{O}(k_{\text{max}}\|\bar{C}_{\text{max}}\|t + \log_2 \epsilon_{\text{QSP}}^{-1}), \quad (31a)$$

$$\|\bar{C}_{\text{max}}\| = k_{\text{max}}^{-1}\|C_{\text{max}}\|. \quad (31b)$$

Here, according to Eq. (18), the maximum value of the Fourier coordinate, k_{max} , scales as

$$k_{\text{max}} = \mathcal{O}(\epsilon_{\text{LCHS}}^{-1}) \quad (32)$$

if one uses the special kernel (8). On the other hand, according to Eq. (17), this scaling is improved exponentially,

$$k_{\text{max}} = \mathcal{O}\left(\log^{1/\beta}(\epsilon_{\text{LCHS}}^{-1})\right), \quad (33)$$

if one applies the kernel (5). For approximate LCHS, the analytic approximations to the LCHS integral allow one to improve the truncation error to

$$k_{\text{max}} = \mathcal{O}(\log(\epsilon_{\text{LCHS}}^{-1})). \quad (34)$$

The QSP error increases with N_k , and, so, in order to have the total QSP error be close to the truncation error ϵ_{LCHS} , the local error ϵ_{QSP} should scale at least as

$$\epsilon_{\text{QSP}} = \mathcal{O}(\epsilon_{\text{LCHS}}/N_k). \quad (35)$$

This results in the following complexity of the LCHS selector

$$\mathcal{O}(k_{\text{max}}\|\bar{C}_{\text{max}}\|t + Q_{\epsilon, \text{QSP}}) \quad (36)$$

where the work for QSP needed to achieve the target LCHS error is

$$Q_{\epsilon, \text{QSP}} := \log_2(N_k \epsilon_{\text{LCHS}}^{-1}) = n_k + \log_2(\epsilon_{\text{LCHS}}^{-1}). \quad (37)$$

Efficient block-encoding is essential for quantum advantage because, for problems with the right structure, this can exponentially reduce the classical complexity. If an efficient block-encoding of the matrices B_m and A_H is provided, then each call to the oracle U_C requires

$$Q_{\text{BE}} = \mathcal{O}(n_k + \text{poly}(n_x, \zeta')) \quad (38)$$

gates where $n_x = \log_2(N_x)$ is the number of qubits in the register r_x required to store the solution vector on N_x points of a spatial grid, and the additive term n_k appears due to the subcircuit 4c computing the sine function.

In our case of the linear advection-diffusion equation with constant coefficients (explored in Sec. IV), the matrices A_H and B_m have a simple two and three-banded diagonal structure, respectively. Hence, $\zeta' \leq 3$, with constant matrix elements, which can be easily encoded by several standard rotation gates, Fig. 5. Therefore, the block-encoding error is defined by the precision of the standard rotation gates R_y and R_z , which is the set to double floating-point precision in our numerical simulations. For both matrices A_H and B_m , we take $\|M\| = \|C_{\text{max}}\|$.

The scaling of the selector S becomes

$$Q_{\text{sel}} = \mathcal{O}(Q_{\text{BE}}[k_{\text{max}}\|\bar{C}_{\text{max}}\|t + Q_{\epsilon, \text{QSP}}]) \quad (39)$$

For the final result, the dependence of k_{max} on ϵ_{LCHS} is described either by Eq. (32), Eq. (33), or by Eq. (34) depending on the chosen LCHS kernel.

4. Comparison to Trotterization

One can compare the complexity scaling of the optimal selector in (39) with the scaling of the selector based on the trotterization of A_H and A_L employed in our previous work 19

$$Q_{\text{trot}} = \mathcal{O}\left(Q_{\text{BE}}\left[\|A_H t\| + \epsilon_{\text{LCHS}}^{-1}\|A_L t\| + Q_{\xi} \log_2 \epsilon_{\text{QSP}}^{-1}\right]\right). \quad (40)$$

In this case, the QSP approximation cost, Q_{ξ} , depends on the trotterization order, p , scaling as $t^{2+2/p}$,

$$Q_{\xi} = \mathcal{O}\left[\left(\frac{\|A_C\|\|A_L t\|^2}{\epsilon_{\text{LCHS}}^3}\right)^{1+1/p}\right]. \quad (41)$$

One can see that the complexity of our new selector, Eq. (39), does not include the multiplicative factor Q_{ξ} which scales poorly with the simulation time. In addition, for time-independent problems, the algorithmic accuracy no longer depends on the trotterization step and, hence, on the commutator of the matrices A_L and A_H .

B. Quadrature Convergence Sets LCHS Complexity

In order to complete the analysis, we must select a numerical method to compute the LCHS integral and estimate N_k by placing bounds on the convergence of the error in computing the value of the integral. First, we consider the prior art^{18–20} available at the time we first proposed our algorithm. We analyze quadrature based on the trapezoidal rule¹⁸ in Sec. III B 1, composite Gaussian quadrature¹⁸ in Sec. III B 2, and the trapezoidal rule after the $\sin(\theta)$ transformation¹⁹. Then, because this field is moving rapidly, in Sec. III B 3, we compare to recent results that consider approximate LCHS^{24,25}. Perhaps, more importantly, in Sec. III B 4 we consider how the improved convergence analysis for the trapezoidal rule based on the analyticity of the kernel²⁴ applies to the case of the near-optimal kernels (5).

In Sec. III B 4, we prove that the sinusoidal transformation recasts the numerical integral as Fejér-Clenshaw-Curtis (FCC) quadrature⁴⁵ which can be computed classically using the fast Fourier transform (FFT). Next, we consider the error convergence and the complexity in this setting. We prove that, for kernels that are analytic on the real line, any numerical integration method that is based on interpolation rather than an exact projection generically has three asymptotic regions of convergence: an analytic region of exponential convergence as assumed in 24 and 25, a p -times differentiable region of power law convergence, and then a final region where convergence halts due to aliasing between nonlinear terms of different orders.

For the first time, our analysis definitively answers the question raised by L. Trefethen 45: *Is Gauss[-Legendre] quadrature better than [Fejér-]Clenshaw-Curtis quadrature?* **The answer is no:** both are Gaussian quadrature methods and both are matched optimally to different forms of the integrand. In particular, FCC quadrature is optimally matched to integrands that are periodic and in C^∞ , i.e. smooth to all orders.

Then, we reconsider the analysis of 24 in light of the optimal convergence of FCC quadrature. We prove that in the analytic region of exponential convergence, FCC quadrature is equivalent to Chebyshev–Gauss–Lobatto (CGL) quadrature, and, like any Gaussian quadrature method, converges twice as fast as when the integration points are not chosen in an optimal fashion. Hence, for fixed $k_{\max}$, FCC converges better than the uniform sampling trapezoidal rule considered in 18, 20, and 24 by a factor of 2. While the rate of convergence is only improved by a constant of order unity, this is the best known constant and can essentially be considered optimal. Moreover, the classical resources needed for the QSVT calculation based on FCC quadrature are optimal over other types of Gaussian quadrature because they can be computed with FFTs.

Our analysis is the first to consider the multiple asymptotic regions that occur in practice as well as the impact of nonlinear aliasing between the two factors in the integrand: the weight function, $w(k) = \xi(k)/(1+ik)$, and the unitary, $e^{iC_j t} = e^{i(A_H + k_j A_L)t}$. The estimates by other authors^{18,20,24} do not account for these facts, and, hence, may be too loose unless care is taken to resolve these issues. First, it is impor-

tant to understand how the choice of $k_{\max}$ impacts which of the three regions of asymptotic convergence that one should consider, as this will impact the dependence of complexity on N_k . Second, it is important to understand that continuing to increase $k_{\max}$ can always improve convergence, and, that, when one is limited by classical resources, e.g. for computing and compiling the QSVT, one may very well need to choose $k_{\max}$ beyond the analytic region (1) of exponential convergence. Third, we derive somewhat pessimistic lower bounds for complexity that are guaranteed to be correct. Fourth, we point out that, since this part of the calculation must be performed classically, one can always estimate convergence by computing the FFT of the LCHS integral of interest.

Finally, while our theoretical understanding of LCHS has greatly improved, our numerical results in Sec. IV have not been modified by the improved analysis. The great virtue of our empirical numerical results is that they allowed us to carefully analyze and understand convergence – proving that it was better than anticipated by the early analysis in 18–20 – before the improved analysis was developed.

1. Prior Art: Trapezoidal Rule

First, consider the analysis of 18, which is based on using the trapezoidal rule for the discretization of the LCHS integral (3), the discretized step in the Fourier space should be small enough to make the discretization error close to the truncation error ϵ_{LCHS} . The error in each subinterval is determined by the second derivative, which leads to the estimate

$$\epsilon_{\text{quad}}^{\text{trap}} = f''(k/k_{\max})(2k_{\max})^3/N_k^2 \quad (42)$$

where $f(k/k_{\max})$ is the integrand in terms of the scaled variable $k/k_{\max}$. This imposes the following condition on N_k (see Sec. II in the Supplemental Material of Ref. 18):

$$N_k = \mathcal{O}\left(k_{\max}^{3/2} \|A_L t\| m_\epsilon^{1/2} \epsilon_{\text{LCHS}}^{-1/2}\right). \quad (43)$$

(Note that, in this equation, we have corrected a typo in the power law for ϵ_{LCHS} reported in 18, Sec. II, Eq. S22.) Clearly, reducing the dependence of $k_{\max}$ on ϵ_{LCHS} from Eq. (32) to Eq. (33) or Eq. (34) will be beneficial in reducing the overall scaling. This yields a selector complexity that scales as

$$Q_{\text{sel}} = \mathcal{O}\left(Q_{\text{BE}} \|\bar{C}_{\max} t\| \log^{1/\beta}(\epsilon_{\text{LCHS}}^{-1})\right). \quad (44)$$

Now, let us consider the complexity scaling of the calculation of the weights. As shown in Eq. 90 in Ref. 19, if the oracle $U_{\sqrt{w}}$ computing the LCHS weights is implemented using QSVT circuits, then the complexity scaling of the weight computation including AA is

$$Q_w = \mathcal{O}\left(n_k \left[k_{\max}^{3/2} \epsilon_w^{-1}\right]\right) = \mathcal{O}\left(n_k \left[k_{\max}^{3/2} m_\epsilon \epsilon_{\text{LCHS}}^{-1}\right]\right), \quad (45)$$

assuming that the error of the weight computation is of the order of ϵ_{LCHS} . While this has worse dependence on precision than the selector, this estimate will be remedied by the improved analysis in the next subsections.

2. Prior Art: Composite Gaussian Quadrature = Trapezoidal Rule + Gauss-Legendre Quadrature

An, et al., 20 analyzed the resources required by near-optimal LCHS based on composite Gauss-Legendre quadrature for numerical integration of the kernel. This is equivalent to a standard hp -finite-element construction of the numerical integration rule, where the number of elements and the quadrature polynomial degree is set by the desired accuracy. The truncation error can be bounded by

$$\epsilon_{\text{trunc}} \leq \left[\int_{-\infty}^{-k_{\max}} + \int_{k_{\max}}^{\infty} \right] |w(k)| dk \quad (46)$$

$$\leq m_{\beta} \int_{k_{\max}}^{\infty} e^{-k^{\beta} \cos(\beta\pi/2)} dk/k \quad (47)$$

$$= M_{\beta} E_1 \left(k \cos^{1/\beta}(\beta\pi/2) \right) \quad (48)$$

$$\leq M_{\beta} e^{-k_{\max}^{\beta} \cos(\beta\pi/2)} \log \left(1 + \frac{1}{k \cos^{1/\beta}(\beta\pi/2)} \right). \quad (49)$$

where $M_{\beta} = m_{\beta}/\beta$. The second line results from 20 Appendix D, Eq. 186, while the final line results from a bound for the exponential integral, $E_1(x)$. Note that this is tighter than the bound reported in 20 Lemma 9 Eq. 62. Thus, in order to reduce the truncation error below threshold, one must choose

$$k_{\max} \geq k_{\text{trunc}} := \frac{\log^{1/\beta} (M_{\beta} m_{\epsilon} / k_{\text{trunc}} \epsilon_{\text{LCHS}})}{\cos^{1/\beta}(\beta\pi/2)}. \quad (50)$$

In order to reach the largest time scales without the Gibbs phenomenon ruining accuracy at the end points of the domain, the minimum value of k must satisfy $k_{\min} \|A_L t\| < \pi$ where $\|\cdot\|$ is the spectral norm.¹⁹ For a numerical study of these issues, please see 19 (Sec. 4.1 and Fig. 5(a)). This yields the lower bound

$$N_k := 2k_{\max}/k_{\min} \geq N_{\text{trunc}} := 2k_{\text{trunc}} \|A_L t\|/\pi. \quad (51)$$

For the method of 20, they state it is sufficient to choose $k_{\min} \|A_L t\| \simeq 1$, and, hence, they use the more restrictive bound $N_k \geq \pi N_{\text{trunc}}$. Overall, N_k scales as

$$N_k = \mathcal{O} \left(\|A_L t\| \log^{1/\beta}(\epsilon_{\text{LCHS}}^{-1}) \right) \quad (52)$$

and the selector complexity scales as

$$Q_{\text{sel}} = \mathcal{O} \left(Q_{\text{BE}} \|\bar{C}_{\max} t\| \log^{1/\beta}(\epsilon_{\text{LCHS}}^{-1}) \right). \quad (53)$$

However, in order to control the quadrature error, 20 (Sec. 3.1) estimated that a Gauss-Legendre quadrature rule of order

$$2n_q > \log \left(k_{\text{trunc}} M'_{\beta} m_{\epsilon} \epsilon_{\text{LCHS}}^{-1} \right) \quad (54)$$

is required, where M'_{β} is another constant. Here, the extra n_q quadrature points are used to provide greater resolution

within each of the N_k subintervals. Thus, the total number of sample points, $N_q = n_q N_{\text{trunc}}$, is 20 (Sec. 3.1)

$$N_q = \mathcal{O} \left(k_{\text{trunc}} \frac{\|A_L t\|}{\pi} \log \left(k_{\text{trunc}} M'_{\beta} m_{\epsilon} \epsilon_{\text{LCHS}}^{-1} \right) \right). \quad (55)$$

This is proportional to the cost of computing the answer classically.

As will be explained more fully in the next subsection, the overall work for the quantum composite Gaussian quadrature computation is only

$$Q_w := n_k N_q / p_w^{1/2} = n_k n_q N_{\text{trunc}} / p_w^{1/2}, \quad (56)$$

which yields

$$Q_w \sim \mathcal{O} \left(n_k (2k_{\text{trunc}})^{3/2} \|A_L t\| \log \left(k_{\text{trunc}} M'_{\beta} m_{\epsilon} \epsilon_{\text{LCHS}}^{-1} \right) \right) \quad (57)$$

where, here, we use their bound $N_k \geq \pi N_{\text{trunc}}$. The extra $(2k_{\text{trunc}})^{1/2}$ increase in complexity over that reported in 20 (Sec. 3.1) is due to the need to improve the success probability with AA. The fact that AA is necessary for this step was clearly demonstrated in our prior numerical studies 19 and in the example below. The overall scaling then becomes

$$Q_w = \bar{\mathcal{O}} \left(n_k \|A_L t\| \log^{1+3/2\beta}(\epsilon_{\text{LCHS}}^{-1}) \right). \quad (58)$$

where the notation $\bar{\mathcal{O}}$ indicates that this expression neglects subdominant logarithmic factors.

3. Contemporary Results: Trapezoidal Rule for Analytic Integrands

Low and Somma 24 present a detailed error analysis based on the two-step process of (1) applying uniform quadrature and (2) approximating the kernel with QSVT. For the first step, their Lemma 10 proves that, when the kernel is analytic in a complex strip of height $|\Im(k)| < a$ and decays uniformly in the strip as $|k| \rightarrow \infty$, uniform quadrature leads to exponential convergence. In this case, the error is bounded as $\epsilon_{\text{LCHS}} < M_a / (e^{2\pi a/k_{\min}} - 1)$, for some constant, $M_a < \hat{M}_a e^{a\|A_L t\|_1}$, where, in this context, $\|A_L\|_1 = \|A_L\|_{L^1}$. All kernels under consideration here, as well as in 18, 20, 24, and 25 have poles at $k = -i$ and some also have poles at $k = i$; hence, one must choose $0 < a < 1$ for all of these cases. Achieving a given error requires

$$\begin{aligned} k_{\min} &\leq 2\pi a / \log(1 + M_a \epsilon_{\text{LCHS}}^{-1}) < 2\pi a / \log(M_a \epsilon_{\text{LCHS}}^{-1}) \\ &= 2\pi / [\|A_L t\|_1 + a^{-1} \log(\hat{M}_a \epsilon_{\text{LCHS}}^{-1})], \end{aligned} \quad (59)$$

and they make the choice $a = 1/2$ in the final line. Thus, the number of integration subintervals, $N_k = 2k_{\max}/k_{\min}$,

$$N_k = k_{\max} [\|A_L t\|_1 + a^{-1} \log(\hat{M}_a \epsilon_{\text{LCHS}}^{-1})] / \pi. \quad (60)$$

Due to the second term, this is somewhat larger than the estimate $N_k \sim \mathcal{O}(k_{\max} \|A_L t\|)$ in the previous subsection. Note,

however, that, according to our study of the Gibbs phenomenon in 19, it would be more accurate to consider the periodic extension of the interval, which implies that one should replace $2\pi \rightarrow \pi$ in (59), and this doubles the lower bound for N_k in (60).

The other key difference is that, for the more optimal approximate LCHS kernel described in 24 (Theorem 2), achieving the truncation error requires

$$k_{\max} \geq k_{\text{trunc}}^{\text{approx-LCHS}} = 2c^{-1} \log(M_c \epsilon_{\text{LCHS}}^{-1}) \quad (61)$$

for some constants M_c and $c \neq 0$. One can apply the integration strategy of 24 to the near-optimal kernel (5) by using the appropriate function. This simply requires one to use the lower bound for $k_{\max}$, given in (50), in the expressions above. This yields the scaling

$$N_k = \mathcal{O}\left(\left[\|A_L t\| + \log(\epsilon_{\text{LCHS}}^{-1})\right] \log^{1/\beta}(\epsilon_{\text{LCHS}}^{-1})\right) \quad (62)$$

and a selector complexity that scales as

$$Q_{\text{sel}} = \mathcal{O}\left(Q_{\text{BE}} \|\tilde{C}_{\max} t\| \log^{1/\beta}(\epsilon_{\text{LCHS}}^{-1})\right). \quad (63)$$

For the second step, they provide a detailed estimate of the cost of preparing the weights via a QSVT-style construction (see 24 Sec. 5.2). The cost scales as the product $Q_w = n_k N_q / p_w^{1/2}$, where n_k is the size of the k -register, N_q is the polynomial order required to achieve the desired quadrature error, and p_w is the success probability of the block-encoding of the kernel function. The analysis is dominated by the considerations of truncating the integral over the real line to a region of size $2k_{\max}$. The success probability can be shown to scale as

$$p_w = \mathcal{O}(1/2k_{\max}), \quad (64)$$

and, hence, the complexity to amplitude amplify this to order unity is $p_w^{-1/2} = \mathcal{O}(2k_{\max})^{1/2}$. We also confirmed this scaling in the numerical examples of 19 and in the example below.

It can also be shown that, as long as the quantum state created has finite variance, the error in the state created by the QSVT weight evaluation is bounded by $\epsilon_w < \mathcal{O}((2k_{\max})^{1/2} \hat{\epsilon}_w)$, and, hence, the target QSVT error must satisfy

$$\hat{\epsilon}_w \leq \mathcal{O}\left(\frac{\epsilon_w}{(2k_{\max})^{1/2}}\right) = \mathcal{O}\left(\frac{\epsilon_{\text{LCHS}}}{m_\epsilon (2k_{\max})^{1/2}}\right). \quad (65)$$

A significant improvement over the previous analysis is the recognition that, if the weight function is analytic, it can be approximated well with a globally convergent Chebyshev-Fourier series. Hence, there is no need to compute multiple QSVT approximations for each subinterval. Let the region of analyticity of the weight function, $w(k)$, in the complex plane be specified by the Bernstein ellipse, Γ ,

$$\begin{aligned} k &= k_x + ik_y = \frac{1}{2}(\rho e^{i\theta} + \rho^{-1} e^{-i\theta}) \\ &= \frac{1}{2}(\rho + \rho^{-1}) \cos(\theta) + i \frac{1}{2}(\rho - \rho^{-1}) \sin(\theta). \end{aligned} \quad (66)$$

In this case, the Bernstein ellipse is determined by

$$\rho - \rho^{-1} = 2a/2k_{\max} = a/k_{\max}, \quad (67)$$

which has the solution

$$\rho = a/2k_{\max} + \sqrt{1 + (a/2k_{\max})^2}. \quad (68)$$

Then, the convergence of the Chebyshev series for $w(k)$ can be bounded via (46 Theorem 8.2)

$$\hat{\epsilon}_w < M_\rho \rho^{1-N_q}/(\rho - 1), \quad (69)$$

where $M_\rho = 4 \max_{\partial\Gamma}(w)$ is determined by the maximum of $w(k)$ on the boundary of the ellipse, $\partial\Gamma$. This bound is reduced by $1/2$ if an exact projection is used instead of interpolation.⁴⁶ Thus, this leads to the requirement

$$N_q - 1 > \frac{\log(M_\rho \hat{\epsilon}_w^{-1}/[\rho - 1])}{\log(\rho)}. \quad (70)$$

Using the Taylor series approximation $\log(\rho^2) \simeq \rho^2 - 1 + \dots$ near $\rho = 1$ and the fact that $\rho - 1 \geq a/2k_{\max}$, it is sufficient to choose

$$N_q - 1 \geq \frac{2k_{\max}}{a} \log\left(\frac{2k_{\max} M_\rho}{a \hat{\epsilon}_w}\right) \quad (71)$$

$$= \frac{2k_{\max}}{a} \log\left(\frac{(2k_{\max})^{3/2} M'_\rho}{a \epsilon_{\text{LCHS}}}\right) \quad (72)$$

where, in the final line, we define $M'_\rho = m_\epsilon M_\rho$. Thus, the overall scaling for $Q_w = n_k N_q / p_w^{1/2}$ is

$$Q_w \geq \mathcal{O}\left(n_k \frac{(2k_{\max})^{3/2}}{a} \log\left(\frac{(2k_{\max})^{3/2} M'_\rho}{a \epsilon_{\text{LCHS}}}\right)\right). \quad (73)$$

This strategy can be applied to the near-optimal kernel in Eq. (5) by using the lower bound for $k_{\max}$ in Eq. (50). This yields a kernel complexity that scales as

$$Q_w = \bar{\mathcal{O}}\left(n_k \log^{1+3/2\beta}(\epsilon_{\text{LCHS}}^{-1})\right) \quad (74)$$

where the notation $\bar{\mathcal{O}}$ indicates that this expression neglects subdominant logarithmic factors. Here, we do not expand n_k because it only depends logarithmically on $\|A_L t\|$ and $\log(\epsilon_{\text{LCHS}}^{-1})$. Relative to Eq. (57), the major improvement is that there is no factor of $\|A_L t\|$.

Note that, because the Chebyshev nodes do not agree with the uniform sample points used for the uniform integration method, the overall effective number of sample points is

$$\begin{aligned} N_q N_k \sim \mathcal{O}\left(\frac{2k_{\max}^2}{a} \left[\|A_L t\| + \frac{1}{a} \log(M_a \epsilon_{\text{LCHS}}^{-1})\right] \right. \\ \left. \times \log\left(\frac{2k_{\max} M'_\rho}{a \epsilon_w}\right)\right). \end{aligned} \quad (75)$$

This represents the complexity scaling of the classical algorithm for computing the integral with this method. Note that this scales similarly to N_q in Eq. (55) for the method of the previous subsection,²⁰ but has worse leading dependence on $k_{\max}$, because information is not reused between the uniform sample points for the integral and the Chebyshev nodes for the weights.

4. Our Method: Fejér-Clenshaw-Curtis Quadrature

Now, let us consider the error convergence for our approach. Using the sinusoidal transformation in Eq. (9) to perform the truncated integral over the real line is equivalent to the use of Fejér-Clenshaw-Curtis (FCC) quadrature over the interval $\theta \in [-\pi/2, \pi/2]$. Thus, we do not need to introduce an additional Gaussian quadrature at each step as in 20. FCC quadrature is known to have exceptionally good convergence properties, often similar to or even rivaling that of Gaussian quadrature.⁴⁵ While much has been written about the mysterious virtues of FCC,⁴⁵ a clear explanation has not been provided until now. Here, we prove that there is a simple explanation for why FCC quadrature performs so well in practice: for a periodic and analytic integrand, it is equivalent to CGL quadrature with the integrand multiplied by $\cos(\theta)$. Thus, relative to 20, our approach clearly reduces the overall complexity of the algorithm as well as the resources required.

The uniform discretization of the angle in Eq. (10) can be interpreted as the evaluation of a Chebyshev quadrature formula over the integrand dk rather than $dk/(1 - (k/k_{\max})^2)^{1/2}$, which is equivalent to Chebyshev quadrature of the integrand multiplied by $(1 - (k/k_{\max})^2)^{1/2}$. In turn, this is equivalent to the sum of a Fourier series of the integrand multiplied by $dk/d\theta = k_{\max} \cos(\theta)$. Hence, FCC quadrature simply uses “uniform quadrature” for the Fourier series, which corresponds to using Chebyshev nodes for the quadrature formula. Given the Fourier series, $f(\theta) = \sum_{m=-\infty}^{\infty} \hat{f}_m e^{im\theta}$, the FCC quadrature rule can be computed analytically

$$\int_{-\pi/2}^{\pi/2} f(\theta) \cos(\theta) d\theta = \sum_{m=-\infty}^{\infty} \frac{\hat{f}_{2m}}{2i(m+1)} + \frac{\hat{f}_{2m}}{2i(m-1)} \quad (76)$$

$$= \sum_{m=-\infty}^{\infty} (-1)^{m-1} \frac{2\hat{f}_{2m}}{(2m)^2 - 1}. \quad (77)$$

A significant advantage of this method is that it can be performed rapidly via fast Fourier transform (FFT) methods^{47,48}; one could even consider the quantum Fourier transform (QFT).⁴⁹

Lemma 2 (FCC quadrature $\equiv$ Fourier quadrature $\equiv$ CGL quadrature). *For the class of functions, $f(\theta) : \mathbb{R} \rightarrow \mathbb{R}$, that are periodic on $\theta \in [0, 2\pi)$ with convergent Fourier series, FCC quadrature of the integrand $\int_{-\pi/2}^{\pi/2} f(\theta) \cos(\theta) d\theta$ is equivalent to integration of the integrand, $f(\theta) \cos(\theta)$, by Fourier series. Applying the coordinate transformation, $z = \sin(\theta)$, proves that both are equivalent to CGL quadrature of the integrand $f(\arcsin(z))(1 - z^2)^{1/2}$, i.e. the integral $\int_{-\pi/2}^{\pi/2} f(\arcsin(z)) dz$, with nodes at the Chebyshev extrema, which are the standard discrete Fourier series nodes. Hence, FCC convergence is set by Fourier series convergence and is equivalent to CGL convergence. Thus, for an N -point method, Fourier harmonics in θ and polynomials in z are integrated exactly up to order $2N - 1$.*

Proof. Using the change of variables $z = \sin(\theta)$ proves that

the continuous integrals

$$\int_{-\pi/2}^{+\pi/2} f(\theta) \cos(\theta) d\theta = \int_{-1}^{+1} f(\arcsin(z)) dz \quad (78)$$

are equal. Using this same change of variables proves that the two discrete approximations to these integrals, FCC quadrature (left) and CGL quadrature (right):

$$\frac{1}{N} \sum_j f(\theta_j) \cos(\theta_j) = \frac{1}{N} \sum_j f(\arcsin(z_j)) (1 - z_j)^2, \quad (79)$$

are equal. This coordinate transformation also proves that FCC quadrature and CGL quadrature at the extrema use the same nodes and weights. Because the Chebyshev-Fourier series for $f(\theta)$ converges, the Chebyshev-Fourier series for $F(\theta) := \int_{-\pi/2}^{\theta} f(\theta') d\theta'$ converges and, hence, the integral $F(\pi/2)$ exists and is finite. Convergence estimates follow from the theory of Gaussian quadrature.⁴⁶ $\square$

The importance of FCC quadrature is that it is optimally adapted to the Chebyshev series used by QSVT. Thus, it can directly use data at the Chebyshev extrema and can converge more rapidly than the uniform quadrature method in 24, which is not matched in an optimal manner. While the form of the LCHS integral used in 20 and 24 is better matched with Gauss-Legendre quadrature, it is not usually recommended to go to exceptionally high order because the numerical calculations of the nodes and weights require exceptionally high precision.⁴⁶ The need to avoid this issue is implicitly recognized in 20 with their choice of composite Gaussian quadrature. In contrast, the FCC quadrature used here does not require an approximate numerical computation of the nodes and weights because it is equivalent to integration by Fourier series, and, hence, can be used to arbitrarily high order.

The accuracy of the kernel function is determined by the accuracy of the QSVT approximation. If the Chebyshev expansion of the kernel is known in closed form, then one has an accurate expression for the Chebyshev coefficients; i.e. using the projection onto Chebyshev polynomials. However, for generic kernels such as (5), the Chebyshev expansion is not known in closed form and the Chebyshev coefficients must be evaluated numerically through interpolation rather than projection. This implies that there is an additional aliasing error due to the fact that the interpolants cause aliasing of polynomial orders that are higher than the finite range of orders included in the calculation.

In this case, the kernels of interest, $w(k) = \xi(k)/(1 + ik) \in L^1$, are meromorphic, analytic on the real line, and decay as $k \rightarrow \pm\infty$. However, once the integral is truncated to the finite domain $k \in [-k_{\max}, +k_{\max})$, the periodic extension of $w(k)$ is no longer smooth to all orders at the boundary. For example, both the Cauchy kernel and the real part of the near-optimal kernels are continuous but not smooth at the boundary, i.e. they are in C^1 , and the imaginary part of the near-optimal kernel is discontinuous at the boundary, i.e. it is in C^0 . Even though the optimal approximate LCHS kernels^{24,25} are analytic, i.e. in C^ω , after truncating the integral to finite $k_{\max}$, the periodic extension of these kernels is no longer in C^ω

and the same issues arise. Yet, thanks to the decay condition, as $k_{\max} \rightarrow \infty$, the “part” of $w(k)$ that is not smooth becomes very small relative to the “part” that is analytic. Since the Fourier series for the kernel is the sum of two parts that have different asymptotics, the sum will generically display multiple asymptotics. These considerations are formalized in the following lemma:

Lemma 3 (Discrete Fourier series can have multiple asymptotics). Consider a function, $w(\theta) : \mathbb{R} \rightarrow \mathbb{R}$, that is periodic on $\theta \in [0, 2\pi)$, p -times differentiable, $w \in C^p$, and has a convergent Fourier series $w(\theta) = \sum_{m=-\infty}^{\infty} \hat{w}_m e^{im\theta}$. Assume the function of interest has the form $w = w_{\infty} + w_p$, where: (1) w_{∞}, w_p are periodic and have convergent Fourier series; (2) $w_{\infty} \in C^{\infty}$ is smooth to all orders; and (3) $w_p \in C^p$ is p -times differentiable. The bandwidth-limited discrete Fourier series coefficients

$$\hat{w}_m := N_q^{-1} \sum_{j=0}^{N_q-1} w(\theta_j) e^{im\theta_j}, \quad (80)$$

where $\theta_j = 2\pi j/N$, can have at least three asymptotics:

1. First, a region of faster than polynomial decay generated by the part of $w(\theta)$ that is in C^{∞} .
2. Second, a region of power law decay of the form $m^{-(p+1)}$ generated by the part of $w(\theta)$ that is in C^p and not in C^{∞} .
3. Third, a region where aliasing completely halts convergence at the Nyquist frequency.

Proof. The sum of the Fourier series coefficients $\hat{w}_m = \hat{w}_{\infty,m} + \hat{w}_{p,m}$ has three components with different asymptotics:

1. Faster than polynomial decay generated by the infinitely-smooth component $w_{\infty} \in C^{\infty}$.
2. Power law decay of the form $m^{-(p+1)}$ generated by the p -times differentiable component $w_p \in C^p$.
3. Aliasing error, due to all harmonics that are higher than the maximum N retained by DFT, halts convergence at the Nyquist frequency.

Depending on the relative size of w_{∞} and w_p , as well as on the resolution, N_q , one or more of these asymptotics will be present and observable. For the assumption $\|w_{\infty}\|_1 \gg \|w_p\|_1$, all three asymptotics are present. If w_{∞} is actually analytic, $w_{\infty} \in C^{\omega}$, then region (1) exhibits exponential decay. $\square$

These three different asymptotics are clearly illustrated in Fig. 6. Clearly, it is best to use the Chebyshev expansion in the region where it is converging exponentially quickly. If higher accuracy is desired, then it is best to add more quadrature nodes; i.e. increase N_q .

For the near-optimal kernels (5) on the periodic domain $k \in [-k_{\max}, k_{\max})$, the power law in region (2) is determined by the fact that the real part has a discontinuous derivative, so that $p = 1$ and the coefficients scale as m^{-2} , and the fact that

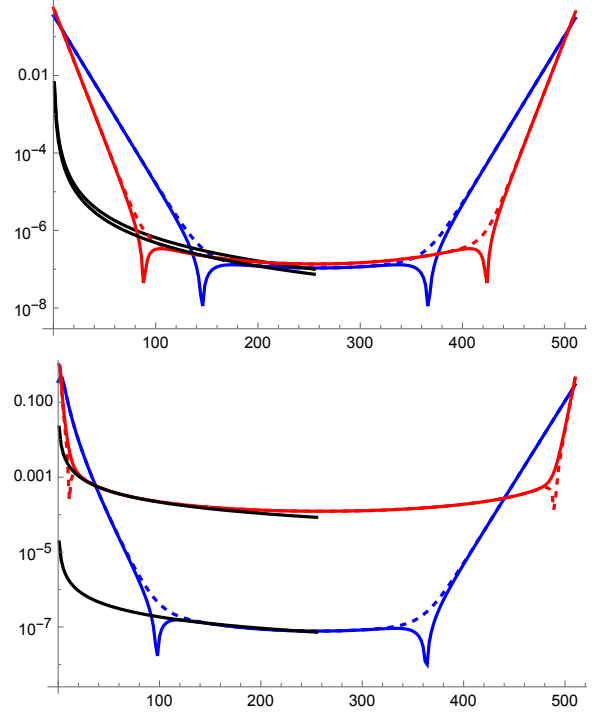


FIG. 6: Absolute value of Chebyshev-Fourier coefficients vs. j before (red) and after (blue) the $\sin(\theta)$ transformation for the: (a) Cauchy kernel (8) and (b) near-optimal kernel (5). Both plots use parameters $\beta = 0.75$, $k_{\max} = 20$, $n_k = 9$, $N_k = 512$ and show the even j coefficients (solid) and the odd j coefficients (dashed) separately. For all cases, the initial exponential decay eventually turns into a power law decay, then halts completely due to aliasing. Fits to the form m^{-p} , where (a) $p = 2$ or (b) $p = 1$ are shown as a guide to the eye (black, solid).

the imaginary part is itself discontinuous, so that $p = 0$ and the coefficients scale as m^{-1} . Because applying the FCC quadrature rule improves convergence by the factor m^{-2} , for the coefficients the sum (79), this improves to m^{-4} for the real part and m^{-3} for the imaginary part. For the Cauchy kernel (8), this region has the same scaling as the real part of the near-optimal kernel. The more optimal kernels proposed in 24 and 25 will have scaling similar to these examples.

It is important to point out that the estimates provided by other authors have not included the asymptotic regions of reduced convergence: (2) and (3). Therefore, the complexity analysis^{20,24,25} may be compromised if one is not careful to ensure that $k_{\max}$ stays within region (1). Clearly, one must take care in ensuring that the power law “noise floor” set by regions (2) and (3) are sufficiently well suppressed to use the estimates that correspond to the analytic part w_{∞} . Yet, for a practical calculation, it would be a mistake to not include regions (2) and (3) when necessary because these regions can always reduce the error. The only thing these regions affect negatively is the complexity analysis and this may not matter for a practical application where one is limited by resolution requirements. This one of the reasons why it is important to perform numerical studies of convergence.

The $\cos(\theta)$ factor has the benefit of making the integrand vanish more quickly at the endpoints $\theta = \pm\pi/2$. In turn, this pushes the error floor, due to non-smooth behavior of the integrand, for a given N_k much further down. Fig. 6(b) shows that this effect is more pronounced for the near-optimal kernel. Unfortunately, the fact that the cosine has the form of a square root, i.e. $\cos(\theta) = (1 - \sin^2(\theta))^{1/2}$, tends to reduce the rate of convergence by a factor of $\mathcal{O}(1)$, which then requires higher $k_{\max}$. In our numerical studies, we found that, to achieve the same error in approximating the weight function, $k_{\max}$ needs to increase by a factor of ~ 2 . However, because our method uses FCC quadrature for quadrature of the LCHS integral, this doubles the rate of convergence by a factor of 2. Thus, there is no actual cost increase, and, for the near-optimal kernels, it appears there is a net benefit. Even if the overall effect was to increase the cost of computing the weights, this has negligible impact on the cost of the selector, which is the most important cost of LCHS. This is because, the part of the query complexity of the selector that is linear in time scales as $n_k = \log_2(N_k) = \log_2(2k_{\max}/k_{\min})$, and, hence only increases logarithmically with $k_{\max}$.

In the region where FCC quadrature is dominated by the smooth component w_∞ , the convergence rate is controlled by CGL quadrature, i.e. improved by a factor of 2 relative to non-Gauss-type methods such as the trapezoidal rule. For N_q -point CGL quadrature of a function, w , analytic within the Bernstein ellipse Γ of radius ρ , which is exact for polynomials of order $2N_q - 1$, the convergence rate is⁴⁶

$$\hat{\epsilon}_w \leq M_{\rho^2} \rho^{2(1-N_q)} / (\rho^2 - 1), \quad (81)$$

where $M_{\rho^2} = 5 \max_{\partial\Gamma}(w)$ is determined by the maximum on the boundary of the ellipse. Thus, this leads to the requirement

$$N_q - 1 \geq \frac{\log(M_{\rho^2} \hat{\epsilon}_w^{-1} / [\rho^2 - 1])}{\log(\rho^2)}. \quad (82)$$

Considering the fact that there are multiple regions of asymptotic convergence, one must take care in the analysis when $k_{\max} > k_{\text{trunc}}$. If we assume analytic convergence within the region of size k_{trunc} , this same rate of convergence may not hold up to $k_{\max}$. Yet, because the error can always be improved by increasing $k_{\max}$, we only demand analyticity within a region of size $|\Re(k)| \leq k_{\text{trunc}}$. This is equivalent to replacing $a/k_{\max} \rightarrow a/k_{\text{trunc}}$ in Eqs. (67) and (68). Hence, the rescaled Bernstein ellipse is determined by

$$\rho - \rho^{-1} = 2a/2k_{\text{trunc}} = a/k_{\text{trunc}}, \quad (83)$$

which has the solution

$$\rho = a/2k_{\text{trunc}} + \sqrt{1 + (a/2k_{\text{trunc}})^2}. \quad (84)$$

Using the Taylor series approximation $\log(\rho^2) \simeq \rho^2 - 1 + \dots$ near $\rho = 1$ and the fact that the Bernstein ellipse satisfies the relation $\rho^2 - 1 \geq a/k_{\text{trunc}}$, implies that it is sufficient to choose

$$N_q \geq 1 + \frac{k_{\text{trunc}}}{a} \log\left(\frac{k_{\text{trunc}} M_{\rho^2}}{a \hat{\epsilon}_w}\right) \quad (85)$$

$$= 1 + \frac{k_{\text{trunc}}}{a} \log\left(\frac{(2k_{\max})^{1/2} k_{\text{trunc}} M'_{\rho^2}}{a \epsilon_{\text{LCHS}}}\right) \quad (86)$$

where, in the final line, $M'_{\rho^2} = m_\epsilon M_{\rho^2}$. To derive this formula, we use the same error requirement as before, $\epsilon_{\text{QSVT}} \leq \epsilon_{\text{LCHS}}/m_\epsilon (2k_{\max})^{1/2}$, because this error is determined by the full range of $\pm k_{\max}$. Clearly, due to the quadratically improved rate of convergence of CGL quadrature over uniform sampling, this method has better scaling than Eq. (70) by a factor $\gtrsim 2$.

Once again, in order to reach the largest time scales via FCC/FFT, we must impose the requirement¹⁹

$$k_{\min} \|A_L t\| = \pi \quad (87)$$

and, hence, the requirement

$$N_k := \frac{2k_{\max}}{k_{\min}} \geq N_{\text{trunc}} := \frac{2k_{\text{trunc}}}{k_{\min}} = 2k_{\text{trunc}} \frac{\|A_L t\|}{\pi}. \quad (88)$$

In order to prevent aliasing between the computation of the weights, $w(k)$, and the Hamiltonian simulation of $e^{ikA_L t}$, for the FCC/FFT method, we must sum the requirements for both, so that $N_k = N_q + N_{\text{trunc}}$. Thus, we arrive at the conclusion

$$N_k > k_{\text{trunc}} \left[\frac{2\|A_L t\|}{\pi} + \frac{1}{a} \log\left(\frac{(2k_{\max})^{1/2} k_{\text{trunc}} M'_{\rho^2}}{a \epsilon_{\text{LCHS}}}\right) \right]. \quad (89)$$

We note that previous reported lower bounds for N_k may be too loose due to aliasing issues.

The bound for N_k can be rephrased in terms of $k_{\max}$ as

$$k_{\max} > k_{\text{trunc}} \left[1 + \frac{\pi}{2a\|A_L t\|} \log\left(\frac{(2k_{\max})^{1/2} k_{\text{trunc}} M'_{\rho^2}}{a \epsilon_{\text{LCHS}}}\right) \right]. \quad (90)$$

Because our results strictly require $k_{\max} > k_{\text{trunc}}$, for FCC quadrature, the extra resolution is always used to increase the range of k and, hence, has the benefit of improving the overall weight, quadrature, and truncation accuracy simultaneously. This is rather different than the quadrature methods of the previous subsections,^{18,20,24} which required subsampling to yield higher resolution within each of the N_k subintervals. For those methods, subsampling was needed to generate high enough accuracy for either the weights or the quadrature error, but did not improve other factors such as the truncation error.

In the final form of (90), we see that it was important to use $|\Re(k)| \leq k_{\text{trunc}}$ as the region of analyticity, rather than $|\Re(k)| \leq k_{\max}$, otherwise this requirement could present a contradiction for sufficiently low ϵ_{LCHS} . Had we used $k_{\max}$ instead of k_{trunc} , we would have arrived at the relation

$$k_{\max} > k_{\text{trunc}} + \frac{\pi k_{\max}}{2a\|A_L t\|} \log\left(\frac{2^{1/2} k_{\max}^{3/2} M'_{\rho^2}}{a \epsilon_{\text{LCHS}}}\right). \quad (91)$$

This leads to the requirement

$$k_{\max} > k_{\text{trunc}} \left[1 - \frac{\pi}{2a\|A_L t\|} \log \left(\frac{2^{1/2} k_{\max}^{3/2} M'_{\rho^2}}{a\epsilon_{\text{LCHS}}} \right) \right]^{-1}. \quad (92)$$

which is only reasonable if the second factor is positive. In turn, this provides a restriction on the requested precision

$$\frac{2\|A_L t\|}{\pi} > \frac{1}{a} \log \left(\frac{2^{1/2} k_{\max}^{3/2} M'_{\rho^2}}{a\epsilon_{\text{LCHS}}} \right). \quad (93)$$

Because the right hand side only increases logarithmically, this inequality may hold true at times that sufficiently large. However, this inequality will always fail for short times at fixed precision. Nevertheless, the result (90) is always valid.

With the FCC method, the total work for preparing the weights quantumly, $Q_w = n_k N_q / p_w^{1/2}$, takes the simple form

$$Q_w \sim \mathcal{O} \left(n_k (2k_{\max})^{1/2} \frac{k_{\text{trunc}}}{a} \log \left(\frac{(2k_{\max})^{1/2} k_{\text{trunc}} M'_{\rho^2}}{a\epsilon_{\text{LCHS}}} \right) \right). \quad (94)$$

This is because the success probability still depends on the total range of $\pm k_{\max}$, so that $p_w = 1/2k_{\max}$. For a classical pseudo-spectral method, one would compute the weights and the nodes from the Chebyshev-Fourier series, multiply the two factors of the integrand in real space, perform the FCC quadrature with the final Chebyshev-Fourier series, and then evaluate the result in real space. Classically, it is efficient to use FFTs and to place the weights and nodes on the same grid of size N_k . Thus, the classical complexity is $3N_k \log(N_k)$ for the three FFTs and $3N_k$ arithmetic operations: N_k multiplications to compose the integrand from the two factors, N_k divisions to perform the quadrature rule, and a final sum of N_k terms to compute the integral.

Quantumly, the FCC complexity in (94) is better than the trapezoidal rule of (73). This is because the factors of $(2k_{\max})^{3/2}$ in (73) are replaced with factors of $(2k_{\max})^{1/2} k_{\text{trunc}}$ in (94), so the main overall effect is an improvement in complexity by the factor $2k_{\max}/k_{\text{trunc}}$. Even for fixed $k_{\max} \sim k_{\text{trunc}}$, the complexity is $2\times$ better than (73). All of these results are improved over (57) because the preparation steps do not scale with time $\mathcal{O}(\|A_L t\|)$.

It is also notable that the classical complexity scaling of an FCC quadrature-based pseudospectral method, $\sim \mathcal{O}(3N_k \log(N_k))$, is better than the results of the previous quadrature methods. Thus, for the practical example studied here, at first, the coordinate transformation appears to require an $\sim 2\times$ larger $k_{\max}$, but this cost increase is then paid back by the use of FCC quadrature, which results in $2\times$ faster convergence of the error in the numerical quadrature rule. Even if this were not the case, the QSVT cost, Q_w , is typically subdominant to the cost of the selector, Q_{sel} , which increases with time.

In terms of precision, the overall selector complexity can be summarized as

$$Q_{\text{sel}} = \mathcal{O} \left(Q_{\text{BE}} \|\bar{C}_{\max} t\| \log^{1/\beta}(\epsilon_{\text{LCHS}}^{-1}) \times \left[1 + \frac{\pi}{2\|A_L t\|} \log(\epsilon_{\text{LCHS}}^{-1}) \right] \right). \quad (95)$$

For example, for the new optimal kernels where $\beta = 1$, the polynomial in this expression is quadratic in $\log(\epsilon_{\text{LCHS}}^{-1})$ but the impact of this term decays in time. The overall kernel complexity can be stated as

$$Q_w = \bar{\mathcal{O}} \left(n_k \log^{1+3/2\beta}(\epsilon_{\text{LCHS}}^{-1}) \times \left[1 + \frac{\pi}{2\|A_L t\|} \log(\epsilon_{\text{LCHS}}^{-1}) \right] \right), \quad (96)$$

neglecting subleading logarithmic factors. Again, we remind the reader that n_k only depends logarithmically on $\|A_L t\|$ and $\log(\epsilon_{\text{LCHS}}^{-1})$. Finally, we arrive at a new theorem for the FCC-based LCHS algorithm.

Theorem 4 (FCC-LCHS Algorithm). *There is a quantum LCHS algorithm based on integration of either an optimal approximate^{24,25} or near-optimal exact²⁰ kernel using FCC quadrature that prepares the normalized solution of Eq. (1) with $\Omega(1)$ success probability and a flag indicating success that uses Q_A queries to the oracle encoding A and Q_{in} queries to the initial state preparation oracle, where*

$$Q_A = \mathcal{O} \left(\frac{\|\psi(0)\|}{\|\psi(t)\|} \|A t\| \log^{1/\beta}(\epsilon_{\text{LCHS}}^{-1}) \right) \quad (97)$$

$$Q_{\text{in}} = \mathcal{O} \left(\frac{\|\psi(0)\|}{\|\psi(t)\|} \right). \quad (98)$$

The parameters are set via the equations: (87) for $k_{\min}$, (89) for N_k , (90) for $k_{\max}$. The gate complexity of the selector is Q_{sel} in (95) and the gate complexity for the kernel weights is Q_w in (96). If an optimal kernel is used, as in 24 and 25, then $\beta = 1$ above and k_{trunc} is set by (61). If a near-optimal kernel is used, as in (5), then k_{trunc} is set by (50).

Proof. The LCHS integral can be approximated optimally using LCU³⁴ where the weight functions are computed optimally using QSVT³⁵ and the success probability is boosted with amplitude amplification³⁶. Using the sinusoidal coordinate transformation in 10, the LCHS integral is computed using the FCC quadrature rule. Given this transformation and an efficient block-encoding of A_H and A_L , the selector can be performed optimally by using a single QSP and qubitization step to compute all required Hamiltonian simulations. The explicit circuit is described in Sec. II C and the complexity analysis is completed in Sec. III B 4. $\square$

C. Further Considerations

1. FCC reduces memory & cost relative to number operator

Another point to consider is that Low & Somma 24 use a diagonal “number operator,” $D = \sum_k k |k\rangle \langle k|$, construction

for the $O_{\sqrt{w}}$ (PREP) operators that is similar to the one they use for the selector S . Relative to the $\sin(\theta)$ encoding, this procedure requires an additional n_k ancillary register and a SWAP of the two registers which costs $\mathcal{O}(3n_k)$ extra 2-qubit gates. If all of these extra ancillary registers can be reused, then the total increase in memory is $2n_k$ and costs at least an additional $2 \times \mathcal{O}(3n_k)$ additional 2-qubit gates. However, if none of these registers can be reused, this requires an ancillary register of size $4n_k$ and costs at least an additional $4 \times \mathcal{O}(3n_k)$ 2-qubit gates. We believe that the $O_{\sqrt{w}}$ (PREP) operators might be able to reuse the extra ancillary register but that the selector cannot, which only requires an intermediate size register $3n_k$ and $3 \times \mathcal{O}(3n_k)$ additional 2-qubit gates. Furthermore, at the end of Sec. IV, we note that this also increases the number of elementary gates needed to represent the single-target-multi-control (STMC) gates that appear in the circuit.

To compare to our approach, even if the coordinate transformation slowed convergence by $4\times$, so that after accounting for the $2\times$ faster convergence of the quadrature error, one would still require N_k to be larger by $2\times$, this means that n_k is only larger by a single qubit.

While these considerations do not change the asymptotic complexity, in practice, it is important to reduce the size of the ancillary register, and, this is one of the benefits of the sinusoidal transformation approach. For the simple example studied here, we found $n_k \approx 8$ offered good resolution, but the approach of 24 might require an ancillary register of 16-32 qubits, which means the total required memory would no longer fit on a single GPU. Near-term quantum computers must also avoid large ancillary registers whenever possible.

2. Remarks on Schrödingerization

For completeness, we point out that, according to the appendices of 24, Schrödingerization effectively uses an asymmetric version of LCU of the form $Sch = O_{\text{left}}^\dagger S O_{\text{right}}$, such that $O_{\text{left},\text{right}}$ prepares a block-encoding of $f_{\text{left},\text{right}}$ subject to the condition $f_{\text{left}}^*(k)f_{\text{right}}(k) = w(k)$. This is suboptimal to the symmetric encoding used in the usual LCU form, $O_{\sqrt{w}}$, because the complexity grows with the subnormalization of the block-encoding. One can prove this using the fact that, for an asymmetric encoding, the cost of Q_w is controlled by the product of the two norms $\|f_{\text{left}}\|_2 \|f_{\text{right}}\|_2$. The cost is minimized for the symmetric encoding, where the norms satisfy $\|f_{\text{left}}\|_2 \|f_{\text{right}}\|_2 = \|w\|_1$.

Yet, we also note that, when using either our method or the method of Ref. 24, the cost increase for Q_w from the QSVT step that constructs the weights is typically benign relative to the cost of the selector, because the cost of the latter must grow with time. If one uses the integration method of Ref. 20 for Schrödingerization, the complexity analysis in Sec. IIIB 2, shows that Q_w also grows with time, so it becomes important to consider the optimal form of LCHS. As mentioned in Sec. III, the same issue occurs if one must use trotterization and this is discussed more fully in 19 and 20.

IV. NUMERICAL SIMULATION RESULTS

To investigate the scaling and the success probability of the described LCHS circuit, we simulate the advection-diffusion equation (ADE):

$$\partial_t \psi = -v \partial_x \psi + D \partial_x^2 \psi \quad (99)$$

with a uniform velocity $v = 1.0$ and diffusivity $D = 0.01$. The detailed quantum circuit for modeling this equation is given in Ref. 29. The initial condition $\psi(0, x)$ is a Gaussian centered at $x = 0.5$ with the width 0.05. The simulated domain, $x = [0, 1)$, with periodic boundary conditions is discretized with N_x spatial points. After the discretization, Eq. (99) is recast as Eq. (1) with the following matrix:

$$A_{i_r i_c} = - \begin{cases} c_{-1}, & i_c = i_r - 1, \\ c_0, & i_c = i_r, \\ c_{+1}, & i_c = i_r + 1, \end{cases} \quad (100)$$

where $i_r, i_c = 0, 1, \dots, (N_x - 1)$, $A_{(N_x-1), N_x} \equiv A_{(N_x-1), 0}$, $A_{0, -1} \equiv A_{0, (N_x-1)}$, and the constant scalars are

$$c_0 = -\frac{2D}{\Delta x^2}, \quad c_{\pm 1} = \left(\frac{D}{\Delta x^2} \mp \frac{v}{2\Delta x} \right), \quad \Delta x = (N_x - 1)^{-1}. \quad (101)$$

For the LCHS simulations, the matrix A is decomposed into Hermitian components A_L and A_H according to Eq. (4), and then one solves Eq. (12) where the LCHS operator U_{LCHS} is represented by the weighted sum (13). The block-encoding oracles computing the matrices A_L and A_H are constructed using the general block-encoding technique for sparse matrices described in Refs. 41 and 44. The corresponding circuits are shown in Fig. 5. There, the register r_x stores the spatial distribution of the variable ψ . The ancillary qubits $a_{x,j}$ are used to address matrix elements either on the main matrix diagonal, or on the matrix left or right sidebands. The ancilla a_e serves as a target qubit for the rotation operators applied for computing the values of the matrix elements. Together, the registers a_x and a_e compose the register a_A in Fig. 4b. In the block-encoding oracle of the matrix A_H , we use a combination of R_y and R_z gates to encode complex values:

$$R_c(\zeta_y, \zeta_z) \equiv R_y(\zeta_y) R_z(\zeta_z). \quad (102)$$

The rotation angles ζ used in the circuits 5 are computed in the following way

$$\zeta_{y,\pm 1} = 2 \arccos(|A_{H,j,j\pm 1}|/\eta_H), \quad (103a)$$

$$\zeta_{z,\pm 1} = -2 \arg(|A_{H,j,j\pm 1}|/\eta_H), \quad (103b)$$

$$\zeta_l = 2 \arccos(|A_{L,j,j+l}|/\eta_{L,l}), \quad l = -1, 0, 1, \quad (103c)$$

where η_H and $\eta_{L,l}$ are real coefficients used to properly normalize the matrix elements $A_{H,j,j\pm 1}$ and $A_{L,j,j+l}$, correspondingly. The exact values of these coefficients can be found in Ref. 29.

The emulation of the resulting LCHS circuit is performed using the QuCF framework²⁸. The total number of qubits in the circuit is

$$n_{\text{LCHS}} = n_x + n_k + n_{\text{BE}} + n_{\text{QSP}} + n_w + n_{\text{AA},w} + n_{\text{init}} \quad (104)$$

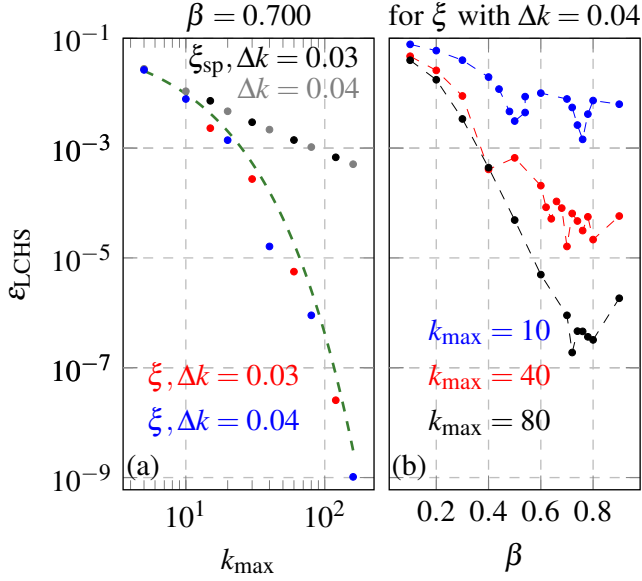


FIG. 7: Results from LCHS simulations of the ADE for $t = 0.8$ without invoking the LCHS circuit, i.e. classical simulations of the LCHS equation (12). (a) The dependence of ϵ_{LCHS} on k_{max} in the LCHS simulations using the special kernel (8) (black and gray markers) and the improved kernel (5) with various Δk and with $\beta = 0.7$ (colored markers). The green dashed line approximates the error with the fitting function (105). (b) The dependence of ϵ_{LCHS} on β in the LCHS simulations with the improved kernel (5) for various k_{max} and for $\Delta k = 0.04$.

where $n_x = \log_2 N_x = 6$ qubits are used for encoding the variable $\psi(x)$, $n_{\text{BE}} = 5$ ancillary qubits are used for block-encoding the matrices C_j , $n_{\text{QSP}} = 2$ ancillae are used for constructing the QSP circuit, $n_w = 1$ ancilla is used for computing the LCHS weights (as a reminder, as discussed in Sec. II C, a brute-force direct computation of the LCHS weights is used in these numerical simulations). Also, $n_{\text{AA},w} = 2$ ancillae are used for AA of the LCHS weights, and $n_{\text{init}} = 2$ ancillae are used for computing the initial conditions. For instance, in the case with $n_k = 12$ discussed below, the total number of qubits in the LCHS circuit is 30.

First of all, we solve Eq. (12) directly without invoking the LCHS circuit 2. The results of these simulations for various LCHS kernels and various values of the scalar β are shown in Fig. 7. In particular, in Fig. 7a, one can see there that the scaling of the truncation error ϵ_{LCHS} with k_{max} can be exponentially improved by using the kernel (5). In this case, the scaling can be approximated reasonably well by the function

$$\epsilon_{\text{LCHS}} \approx 0.12 \exp(-0.5 k_{\text{max}}^\beta), \quad (105)$$

which confirms the theoretical scaling (17). The approximate numerical value observed for the exponent ≈ 0.5 is close to the bound in Eq. (46) which predicts $\cos(\beta\pi/2) \approx 0.454$ for $\beta = 0.7$. Note that, using the bound of Eq. 62 in 20 would have resulted in 1/2 this value, $\cos(\beta\pi/2)/2 \approx 0.227$, which is clearly ruled out by the numerical data. According to

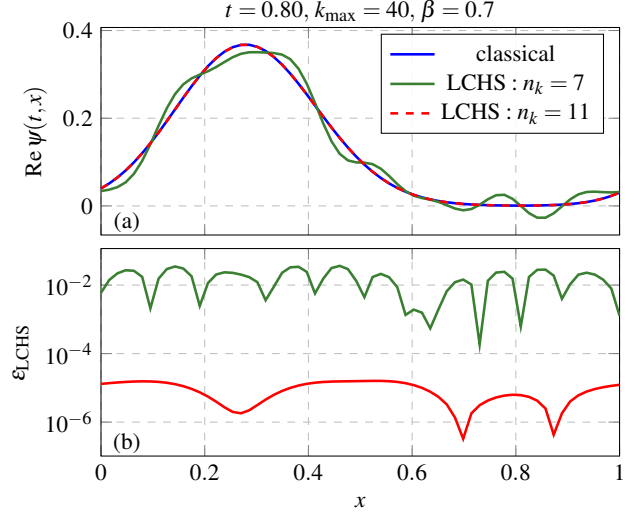


FIG. 8: (a) A comparison between the exact classical simulation (blue line) and the approximate simulations using the LCHS circuit 2 for various n_k (green and red lines). (b) The error of the LCHS simulations.

Fig. 7b, for these choices of parameters, such as D or v , the error is minimized by choosing β in the interval between 0.7 and 0.8, which is consistent with the numerical results of Ref. 20.

Now, to test the LCHS method mapped on the LCU circuit 2 using the coordinate transformation (9) and the kernel (5), we simulate this circuit with different k_{max} and n_k . The circuit is constructed using single-target multicontrolled (STMC) gates such as X , H , R_y , and R_z gates controlled by multiple qubits. According to Refs. 50–52, an STMC gate controlled by n_c qubits can be represented by a circuit with $\mathcal{O}(n_c)$ elementary gates without using ancillae.

The signal $\psi(t, x)$ at $t = 0.8$ and the corresponding error in the LCHS simulation with the circuit 2 are shown in Fig. 8. In particular, one can see that it is possible to decrease the error up to around 10^{-5} using $k_{\text{max}} = 40$ with $n_k = 11$. The scaling of the LCHS error, the number of STMC gates in the circuit, and the circuit success probability are shown in Fig. 9 as functions of k_{max} , n_k , and the simulated time interval t . Clearly, by increasing k_{max} and n_k for a given time instant t , one can exponentially increase the LCHS precision. At the same time, the number of gates in the LCHS circuit increases linearly with k_{max} and only logarithmically with N_k , while the success probability of the circuit stays near the same level. This is consistent with Eqs. (38) and (39). On the other hand, the circuit success probability decreases with time because the simulated signal $\psi(t, x)$ decays in time due to the imposed diffusivity D . Finally, the number of gates in the circuit grows linearly with time while the LCHS precision stays at nearly the same level. If we take into account the decomposition of the circuit into elementary operators^{50–52}, the number of elementary gates should scale at least as $\mathcal{O}(N_{\text{gates}}(n_x + n_k))$ where N_{gates} is the number of STMC gates shown in Fig. 9, and we make a conservative estimate that each STMC gate is controlled by at least $n_x + n_k$

qubits. The actual number of elementary gates will strongly depend on the available set of native gates on a chosen quantum device, as well as on the topology and qubit interconnectivity of the quantum hardware. Here, we point out that, if the size of the ancillary register is increased by $[2 - 4] \times n_k$, as would occur for the approach of 24, then our estimate for the cost of these STMC gates would need to increase by a similar amount, e.g. if the register is size $3n_k$, then the estimate for the cost of the STMC gates increases to $\mathcal{O}(N_{\text{gates}}(n_x + 3n_k))$.

Thus, we demonstrated through actual numerical emulation of the LCHS circuit that our block-encoding technique, Fig. 4, preserves all theoretical benefits of the LCHS algorithm and simplifies its mapping onto a quantum circuit. In particular, our numerical simulations prove that the analytical results of the seminal works 18 and 20 about the high success probability of the LCHS algorithm, the exponentially fast error convergence, and the linear scaling with time all hold for our highly efficient explicit implementation.

V. CONCLUSIONS

In this work, an efficient quantum algorithm based on the Linear Combination of Hamiltonian Simulations (LCHS) has been proposed for simulating dissipative initial-value problems. In this method, a nonunitary operator represented by an exponential function of a non-Hermitian generator is approximated by a weighted sum of Hamiltonian evolutions that depend on the Hermitian components of the generator and on an additional Fourier coordinate.

By recasting the Fourier coordinate as a trigonometric function, we derived a highly efficient encoding of the LCHS sum into a quantum circuit. This enables the use of Fejér-Clenshaw-Curtis (FCC) quadrature for LCHS, which is equivalent to Chebyshev-Gauss-Lobatto (CGL) quadrature. As proven in Theorem 4, this method has the best convergence of all proposed quadrature methods for this purpose. It also allows one to eliminate trotterization and perform all Hamiltonian simulations with a single QSP circuit. In turn, this improves the complexity scaling of the selector in Eq. 39 to the optimal value of linear in time. It also significantly improves the complexity of the weight computation in Eq. 94, as this now no longer increases linearly with time as it does in 19 and 20. These results can be recast in terms of precision in the form of Eq. 95 for the selector and Eq. 96 for the weights. Note, however, that for dissipative evolution, the decay of the solution in time causes an avoidable need to boost the success probability, and, hence, amplifies the cost by the factor $\|\psi(0)\|/\|\psi(t)\|$ unless one performs a spectral shift to eliminate the decay. Finally, we proved that this method can also be applied to the recently proposed approximate LCHS methods,^{24,25} and this would improve the scaling of our algorithm with error from the near-optimal value of $\log^{1/\beta}(\epsilon_{\text{LCHS}}^{-1})$, where $\beta \in [0.7, 0.8]$, i.e. $1/\beta \in [1.25, 1.43]$, to the optimal value of $\log(\epsilon_{\text{LCHS}}^{-1})$. Testing the FCC-LCHS method on these new kernels is an important topic for future work.

The quantum circuit was tested by simulating the

advection-diffusion equation with a uniform velocity and diffusivity, and the numerical simulations confirmed the scaling of the proposed circuit. The proposed encoding of dissipative problems can be used for solving a wide class of nonunitary differential equations including the Liouville equation and various linear embedding techniques of nonlinear problems.

We summarize that, relative to 24, our method achieves benefits in memory and computational cost by eliminating the need for a larger ancillary register of size $[2 - 4] \times n_k$ and at least $[2 - 4] \times \mathcal{O}(3n_k)$ additional 2-qubit gates. That method also increases the cost estimate for the overall number of elementary gates required to represent the single-target-multicontrolled (STMC) gates in the circuit, again rising with the increase in the size of the ancillary k -register.

It should be straightforward to extend our method to the case of systems with a time-dependent generator and inhomogeneous forcing. It is also important to understand the numerical convergence of LCHS for more complicated inhomogeneous differential equations with spatially-dependent and time-dependent coefficients. We plan to explore these interesting questions in future work.

ACKNOWLEDGMENTS

We would like to sincerely thank an anonymous referee for asking us to compare our work to that of two manuscripts 24 and 25 that were posted to arXiv after the submission of our work to the journal. These works are the first we are aware of to investigate optimal approximate LCHS methods and are based on an analytic continuation approach similar to that first proposed for approximating exponential decay in 33. In particular, our extension of the theorems in 24 to near-optimal kernels allowed us to significantly improve our original convergence estimates, based on 18–20. To cleanly separate the priority of the different ideas in this paper, the additional error and complexity analysis that we performed after our original submission is almost entirely presented in Sec. III B.

This work, LLNL-JRNL-2001345, was supported by the U.S. Department of Energy (DOE) Office of Fusion Energy Sciences “Quantum Leap for Fusion Energy Sciences” Project No. FWP-SCW1680 at Lawrence Livermore National Laboratory (LLNL). Work was performed under the auspices of the U.S. DOE under LLNL Contract DE-AC52-07NA27344. This research used resources of the National Energy Research Scientific Computing Center, a DOE Office of Science User Facility supported by the Office of Science of the U.S. Department of Energy under Contract No. DE-AC02-05CH11231 using NERSC award FES-ERCAP0028618.

¹H. Krovi, Improved quantum algorithms for linear and nonlinear differential equations, *Quantum* 7 (2023) 913. doi:10.22331/q-2023-02-02-913.
URL <http://dx.doi.org/10.22331/q-2023-02-02-913>

²N. Linden, A. Montanaro, C. Shao, Quantum vs. classical algorithms for solving the heat equation, *Communications in Mathematical Physics* 395 (2) (2022) 601–641.

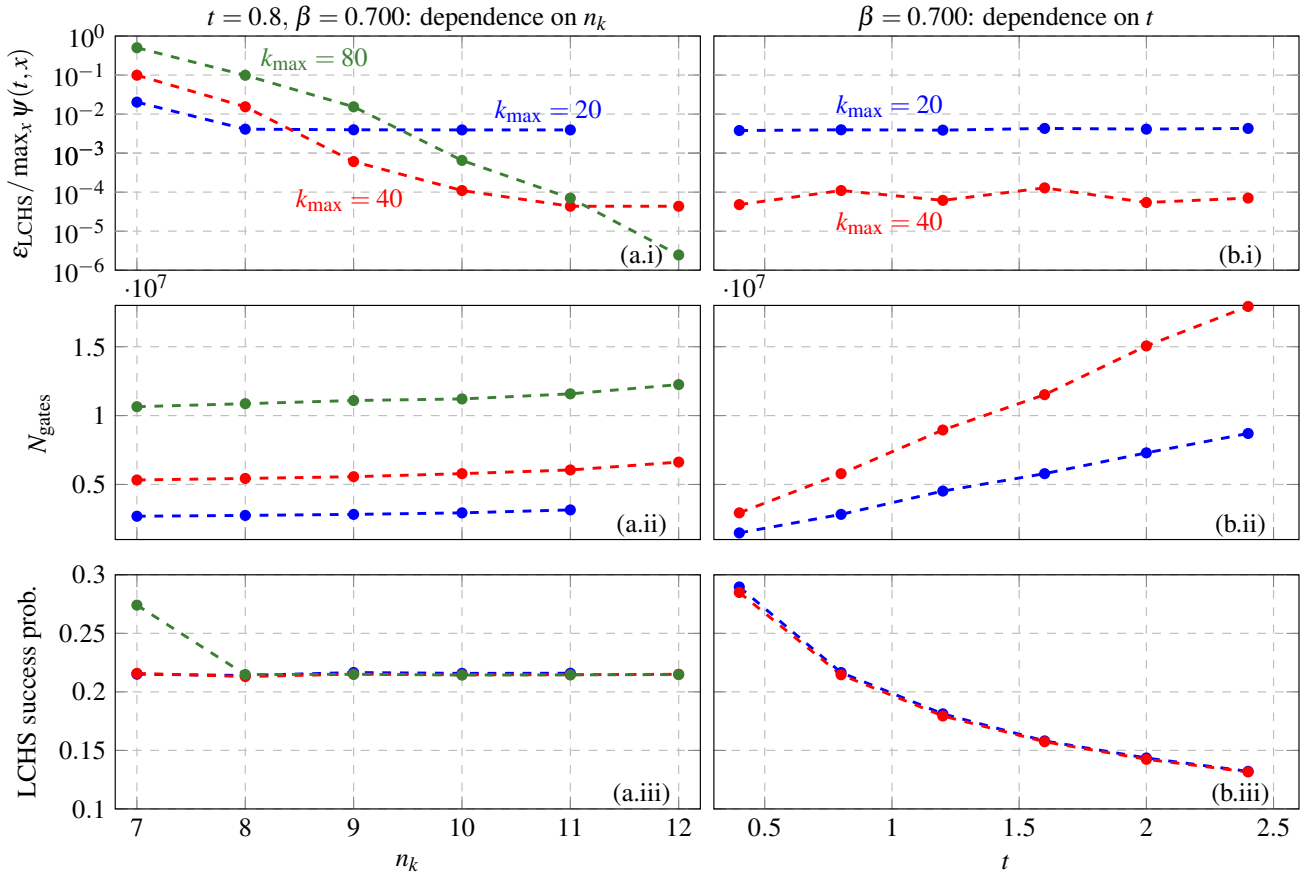


FIG. 9: The dependence of the normalized LCHS error (a.i), the number of STMC gates (a.ii), and the success probability of the LCHS circuit (a.iii) on n_k in the LCHS simulations using the circuit 2 for various $k_{\max}$. The dependence of the error (b.i), STMC gate count N_{gates} (b.ii), and success probability (b.iii) on t for various $k_{\max}$. For $k_{\max} = 20$, the cases with $t = 0.4$ and 0.8 have $n_k = 9$, the cases with $t = 1.2$ and 1.6 have $n_k = 10$, and the cases with $t = 2.0$ and 2.4 have $n_k = 11$. For $k_{\max} = 40$, the corresponding values of n_k are increased by one.

³A. Ambainis, Variable time amplitude amplification and quantum algorithms for linear algebra problems, in: C. Dürr, T. Wilke (Eds.), 29th International Symposium on Theoretical Aspects of Computer Science (STACS 2012), Vol. 14 of Leibniz International Proceedings in Informatics (LIPIcs), Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 2012, pp. 636–647. doi:10.4230/LIPIcs.STACS.2012.636.

URL <https://drops.dagstuhl.de/opus/volltexte/2012/3426>
⁴A. M. Childs, R. Kothari, R. D. Somma, Quantum algorithm for systems of linear equations with exponentially improved dependence on precision, SIAM Journal on Computing 46 (6) (2017) 1920–1950. arXiv:https://doi.org/10.1137/16M1087072, doi:10.1137/16M1087072. URL <https://doi.org/10.1137/16M1087072>

⁵S. Chakraborty, A. Gilyén, S. Jeffery, The power of block-encoded matrix powers: Improved regression techniques via faster hamiltonian simulation, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019. doi:10.4230/LIPIcs.ICALP.2019.33. URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2019.33>

⁶D. An, L. Lin, Quantum linear system solver based on time-optimal adiabatic quantum computing and quantum approximate optimization algorithm, ACM Transactions on Quantum Computing 3 (2) (Mar. 2022). doi:10.1145/3498331. URL <https://doi.org/10.1145/3498331>

⁷P. C. Costa, D. An, Y. R. Sanders, Y. Su, R. Babbush, D. W. Berry, Optimal scaling quantum linear-systems solver via discrete adiabatic theorem, PRX Quantum 3 (2022) 040303.

doi:10.1103/PRXQuantum.3.040303.

URL <https://link.aps.org/doi/10.1103/PRXQuantum.3.040303>

⁸D. Fang, L. Lin, Y. Tong, Time-marching based quantum solvers for time-dependent linear differential equations, Quantum 7 (2023) 955. doi:10.22331/q-2023-03-20-955.

URL <https://doi.org/10.22331/q-2023-03-20-955>

⁹P. Over, S. Bengoechea, P. Brearley, S. Laizet, T. Rung, Quantum algorithm for the advection-diffusion equation by direct block encoding of the time-marching operator, Phys. Rev. A 112 (2025) L010401. doi:10.1103/d8hb-fv93.

URL <https://link.aps.org/doi/10.1103/d8hb-fv93>

¹⁰S. S. Bharadwaj, K. R. Sreenivasan, Compact quantum algorithms for time-dependent differential equations (2024). arXiv:2405.09767. URL <https://arxiv.org/abs/2405.09767>

¹¹C. Sanavio, E. Mauri, S. Succi, Explicit quantum circuit for simulating the advection–diffusion–reaction dynamics, IEEE Transactions on Quantum Engineering 6 (2025) 1–12. doi:10.1109/tqe.2025.3544839. URL <http://dx.doi.org/10.1109/TQE.2025.3544839>

¹²R. Demirdjian, D. Gunlycke, C. A. Reynolds, J. D. Doyle, S. Tafur, Variational quantum solutions to the advection–diffusion equation for applications in fluid dynamics, Quantum Information Processing 21 (9) (2022) 322. doi:10.1007/s11128-022-03667-7. URL <https://doi.org/10.1007/s11128-022-03667-7>

¹³J. Ingelmann, S. S. Bharadwaj, P. Pfeffer, K. R. Sreenivasan, J. Schumacher, Two quantum algorithms for solving the one-dimensional advection–diffusion equation, Computers & Fluids 281 (2024) 106369.

- doi:10.1016/j.compfluid.2024.106369.
URL <http://dx.doi.org/10.1016/j.compfluid.2024.106369>
- ¹⁴S. Jin, N. Liu, Y. Yu, Quantum simulation of partial differential equations via Schrödingerisation (2022). arXiv:2212.13969.
URL <https://arxiv.org/abs/2212.13969>
- ¹⁵S. Jin, N. Liu, Y. Yu, Quantum simulation of partial differential equations: Applications and detailed analysis, *Phys. Rev. A* 108 (2023) 032603. doi:10.1103/PhysRevA.108.032603.
URL <https://link.aps.org/doi/10.1103/PhysRevA.108.032603>
- ¹⁶J. Hu, S. Jin, N. Liu, L. Zhang, Quantum Circuits for partial differential equations via Schrödingerisation, *Quantum* 8 (2024) 1563. doi:10.22331/q-2024-12-12-1563.
URL <https://doi.org/10.22331/q-2024-12-12-1563>
- ¹⁷Z. Lu, Y. Yang, Quantum computing of reacting flows via Hamiltonian simulation, *Proceedings of the Combustion Institute* 40 (1) (2024) 105440. doi:<https://doi.org/10.1016/j.proci.2024.105440>.
URL <https://www.sciencedirect.com/science/article/pii/S1540748924002487>
- ¹⁸D. An, J.-P. Liu, L. Lin, Linear combination of hamiltonian simulation for nonunitary dynamics with optimal state preparation cost, *Phys. Rev. Lett.* 131 (2023) 150603. doi:10.1103/PhysRevLett.131.150603.
URL <https://link.aps.org/doi/10.1103/PhysRevLett.131.150603>
- ¹⁹I. Novikau, I. Joseph, Quantum algorithm for the advection-diffusion equation and the Koopman-von Neumann approach to nonlinear dynamical systems, *Computer Physics Communications* 309 (2025) 109498. doi:<https://doi.org/10.1016/j.cpc.2025.109498>.
URL <https://www.sciencedirect.com/science/article/pii/S0010465525000013>
- ²⁰D. An, A. M. Childs, L. Lin, Quantum algorithm for linear non-unitary dynamics with near-optimal dependence on all parameters, *Communications in Mathematical Physics* 407 (1) (Dec. 2025). arXiv:2312.03916, doi:10.1007/s00220-025-05509-w.
URL <http://dx.doi.org/10.1007/s00220-025-05509-w>
- ²¹I. Joseph, Semiclassical theory and the Koopman-von Hove equation, *Journal of Physics A: Mathematical and Theoretical* 56 (48) (2023) 484001. doi:10.1088/1751-8121/ad0533.
URL <http://dx.doi.org/10.1088/1751-8121/ad0533>
- ²²I. Joseph, Koopman-von Neumann approach to quantum simulation of nonlinear classical dynamics, *Phys. Rev. Res.* 2 (2020) 043102. doi:10.1103/PhysRevResearch.2.043102.
URL <https://link.aps.org/doi/10.1103/PhysRevResearch.2.043102>
- ²³I. Joseph, Y. Shi, M. D. Porter, A. R. Castelli, V. I. Geyko, F. R. Graziani, S. B. Libby, J. L. DuBois, Quantum computing for fusion energy science applications, *Physics of Plasmas* 30 (1) (Jan. 2023). doi:10.1063/5.0123765.
URL <http://dx.doi.org/10.1063/5.0123765>
- ²⁴G. H. Low, R. D. Somma, Optimal quantum simulation of linear non-unitary dynamics (2025). arXiv:2508.19238.
URL <https://arxiv.org/abs/2508.19238>
- ²⁵S. Jin, N. Liu, C. Ma, Y. Peng, Y. Yu, On the Schrödingerization method for linear non-unitary dynamics with optimal dependence on matrix queries (2025). arXiv:2505.00370.
URL <https://arxiv.org/abs/2505.00370>
- ²⁶Y. Sato, H. Tezuka, R. Kondo, N. Yamamoto, Quantum algorithm for partial differential equations of nonconservative systems with spatially varying parameters, *Phys. Rev. Appl.* 23 (2025) 014063. doi:10.1103/PhysRevApplied.23.014063.
URL <https://link.aps.org/doi/10.1103/PhysRevApplied.23.014063>
- ²⁷T. Jones, A. Brown, I. Bush, S. C. Benjamin, QuEST and high performance simulation of quantum computers, *Scientific Reports* 9 (1) (2019) 10736. doi:10.1038/s41598-019-47174-9.
URL <https://doi.org/10.1038/s41598-019-47174-9>
- ²⁸QuCF framework, <https://github.com/QuCF/QuCF> (2024).
- ²⁹Near-optimal LCHS quantum circuits, <https://github.com/QuCF/QuCF/wiki/OPT%E2%80%9090LCHS> (2024).
- ³⁰G. H. Low, I. L. Chuang, Optimal Hamiltonian simulation by quantum signal processing, *Physical Review Letters* 118 (2017) 010501. doi:10.1103/PhysRevLett.118.010501.
URL <https://link.aps.org/doi/10.1103/PhysRevLett.118.010501>
- ³¹A. Gilyén, Y. Su, G. H. Low, N. Wiebe, Quantum singular value transformation and beyond: Exponential improvements for quantum matrix arithmetics, in: *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, STOC 2019, Association for Computing Machinery, New York, NY, USA, 2019*, p. 193–204. doi:10.1145/3313276.3316366.
URL <https://doi.org/10.1145/3313276.3316366>
- ³²D. W. Berry, A. M. Childs, A. Ostrander, G. Wang, Quantum algorithm for linear differential equations with exponentially improved dependence on precision, *Communications in Mathematical Physics* 356 (3) (2017) 1057–1081. doi:10.1007/s00220-017-3002-y.
URL <https://doi.org/10.1007/s00220-017-3002-y>
- ³³T. de Lima Silva, L. Borges, L. Aolita, Fourier-based quantum signal processing (2022). arXiv:2206.02826.
URL <https://arxiv.org/abs/2206.02826>
- ³⁴A. M. Childs, N. Wiebe, Hamiltonian simulation using linear combinations of unitary operations, *Quantum Info. Comput.* 12 (11–12) (2012) 901–924.
- ³⁵G. H. Low, I. L. Chuang, Hamiltonian simulation by qubitization, *Quantum* 3 (2019) 163. doi:10.22331/q-2019-07-12-163.
URL <https://doi.org/10.22331/q-2019-07-12-163>
- ³⁶G. Brassard, P. Høyer, M. Mosca, A. Tapp, Quantum amplitude amplification and estimation, *Quantum Computation and Information* 305 (2002) 53–74. doi:10.1090/conm/305/05215.
URL <http://dx.doi.org/10.1090/conm/305/05215>
- ³⁷G. H. Low, N. Wiebe, Hamiltonian simulation in the interaction picture (2019). arXiv:1805.00675.
- ³⁸S. Jin, N. Liu, C. Ma, On schrödingerization based quantum algorithms for linear dynamical systems with inhomogeneous terms (2024). arXiv:2402.14696.
URL <https://arxiv.org/abs/2402.14696>
- ³⁹S. McArdle, A. Gilyén, M. Berta, Quantum state preparation without coherent arithmetic, arXiv preprint arXiv:2210.14892 (2022).
- ⁴⁰C. F. Kane, S. Hariprakash, N. S. Modi, M. Kreshchuk, C. W. Bauer, Block encoding bosons by signal processing, *Quantum* 9 (2025) 1747. doi:10.22331/q-2025-05-15-1747.
URL <https://doi.org/10.22331/q-2025-05-15-1747>
- ⁴¹I. Novikau, E. A. Startsev, I. Y. Dodin, Quantum signal processing for simulating cold plasma waves, *Phys. Rev. A* 105 (2022) 062444. doi:10.1103/PhysRevA.105.062444.
URL <https://link.aps.org/doi/10.1103/PhysRevA.105.062444>
- ⁴²N. Guseynov, X. Huang, N. Liu, Gate construction of block-encoding for hamiltonians needed for simulating partial differential equations, *Phys. Rev. Res.* 7 (2025) 033100. doi:10.1103/xlpd-fb1g.
URL <https://link.aps.org/doi/10.1103/xlpd-fb1g>
- ⁴³I. Novikau, I. Dodin, E. Startsev, Simulation of linear non-hermitian boundary-value problems with quantum singular-value transformation, *Phys. Rev. Appl.* 19 (2023) 054012. doi:10.1103/PhysRevApplied.19.054012.
URL <https://link.aps.org/doi/10.1103/PhysRevApplied.19.054012>
- ⁴⁴I. Novikau, I. Dodin, E. Startsev, Encoding of linear kinetic plasma problems in quantum circuits via data compression, *Journal of Plasma Physics* 90 (4) (2024) 805900401. doi:10.1017/S00222377824000795.
- ⁴⁵L. N. Trefethen, Is Gauss quadrature better than Clenshaw–Curtis?, *SIAM review* 50 (1) (2008) 67–87.
- ⁴⁶L. N. Trefethen, *Approximation theory and approximation practice*, extended edition, SIAM, 2019.
- ⁴⁷W. M. Gentleman, Implementing clenshaw-curtis quadrature, i methodology and experience, *Communications of the ACM* 15 (5) (1972) 337–342.
- ⁴⁸W. M. Gentleman, Implementing clenshaw-curtis quadrature, ii computing the cosine transformation, *Communications of the ACM* 15 (5) (1972) 343–346.
- ⁴⁹J. Chen, E. Stoudenmire, S. R. White, Quantum fourier transform has small entanglement, *PRX Quantum* 4 (4) (2023) 040318.
- ⁵⁰A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, H. Weinfurter, Elementary gates

- for quantum computation, *Phys. Rev. A* 52 (1995) 3457–3467. doi:10.1103/PhysRevA.52.3457.
 URL <https://link.aps.org/doi/10.1103/PhysRevA.52.3457>
- ⁵¹B. Claudon, J. Zylberman, C. Feniou, F. Debbasch, A. Peruzzo, J.-P. Piquemal, Polylogarithmic-depth controlled-not gates without ancilla qubits, *Nature Communications* 15 (1) (2024) 5886. doi:10.1038/s41467-024-50065-x.
 URL <https://doi.org/10.1038/s41467-024-50065-x>
- ⁵²N. Guseynov, N. Liu, Efficient explicit circuit for quantum state preparation of piecewise continuous functions, *Phys. Rev. A* 113 (2026) 012604. arXiv:<https://arxiv.org/abs/2411.01131>, doi:10.1103/plc3-2jyx.
 URL <https://link.aps.org/doi/10.1103/plc3-2jyx>